

文档编号	TY-SUM-20231030A
版 本	A
页 数	50 页

科学计算环境 APP 开发用户手册

科学计算与系统建模仿真项目组

编制	张新城	生效日期	2023-10-30
审核	郭俊峰	批准	刘奇

文件变更摘要

日期	版本	变更说明	修订	审核	批准
2023/10/30	V1.0	初始建立	张新城	郭俊峰	刘奇

苏州同元软控信息技术有限公司版权所有，未经许可不得复制、传播或以其他方式使用；
版权所有，侵权必究。

目 录

1. 概述	1
1.1 什么是 APP	1
1.2 APP 的开发运行模式	1
1.2.1 运行模式	1
1.2.2 开发运行流程	4
2. 如何快速开发 APP	9
2.1 曲线拟合 APP 介绍	9
2.2 APP 开发测试打包	9
2.3 APP 安装使用	9
2.3.1 可视化管理	10
2.3.2 命令式管理	21
3. 综合案例介绍	25
3.1 Python 版 APP	25
3.1.1 APP 开发	25
3.1.2 APP 测试	35
3.1.3 APP 打包	37
3.2 C++版 APP	39
3.2.1 APP 开发	39
3.2.2 APP 测试	48
3.2.3 APP 打包	49

1. 概述

1.1 什么是 APP

APP 是带交互界面的应用程序，提供面向特定场景的专业应用，如控制系统设计与分析应用。APP 通常依赖函数库或模型库，具备 GUI 实现交互入口，通过专业算法调用底层函数。APP 作为专业工具，需要在 Syslab 平台的基础计算能力上，构建面向特定应用的专业计算能力。

1.2 APP 的开发运行模式

1.2.1 运行模式

APP 运行过程中通过 APP SDK 与 Syslab 平台交互。Syslab 平台提供了多种 APP SDK，以支持多种图形应用开发平台（PyQt、C++/Qt、JavaScript 等）来开发 APP，包括 Python 版本 SDK、C++版本 SDK、Javascript 版本 SDK。

APP SDK 提供了 APP 通信 API，可实现 APP 与 Syslab 平台之间数据交互和功能调用，包括 APP 从 Syslab 工作空间中获取数据，APP 将数据写入 Syslab 工作空间，APP 调用 Syslab 执行科学计算等。

下文将阐述 APP 运行的原理。

1.2.1.1 组件关系图

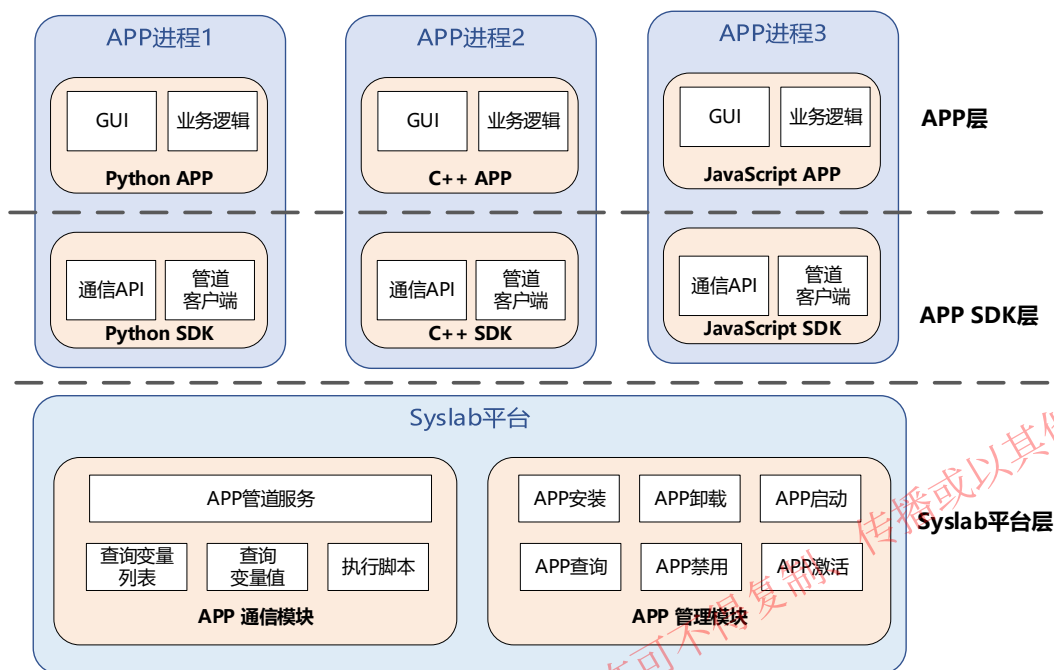


图 1-1 APP 组件关系图

- **APP 层:** 负责开发 GUI 图形用户界面和 APP 的业务逻辑。用户可以使用主流的图形应用开发平台（PyQt、C++/Qt、JavaScript 等）来开发 APP，并通过使用 APP SDK 来实现与 Syslab 平台集成和通信。
- **APP SDK 层:** 负责 APP 与 Syslab 平台之间的通信，实现了进程间通信的管道客户端，并提供了 APP 通信 API。Syslab 提供多款 APP SDK，包括 Python SDK、C++ SDK、JavaScript SDK 等，便于用户快速开发。
- **Syslab 平台层:** 包含 APP 通信与 APP 管理两个模块。APP 通信模块提供了 APP 管道服务，提供了查询变量、执行脚本等服务化能力。APP 管理模块提供了对 APP 进行注册安装、卸载、启动、查询、禁用、激活等功能，实现对 APP 的全生命周期管理。

1.2.1.2 运行时序图

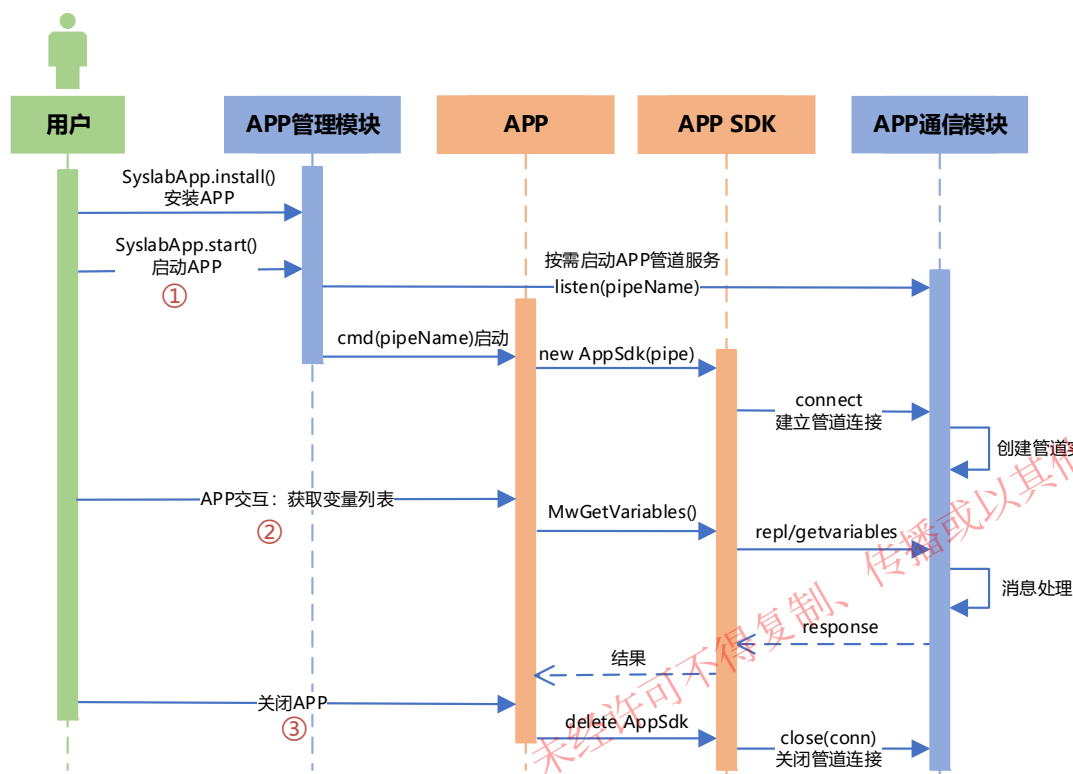


图 1-2 APP 运行时序图

APP 的运行过程主要包含以下 3 个部分：

- 启动 APP

- APP 注册安装到 Syslab 平台后，用户从 Syslab 平台启动 APP
- 启动 APP 时 Syslab 平台先启动 APP 管道服务端
- Syslab 通过 cmd 命令方式启动 APP 进程，并将管道名以参数方式传递给 APP 进程
- APP 调用 APP SDK 在 APP 进程内启动 APP 管道客户端，与 Syslab 建立管道连接

- APP 交互

- 用户在 APP 的图形界面操作，以获取 Syslab 变量列表为例，APP 调用 APP SDK 的获取变量列表 API: MwGetVariables
- APP SDK 向 Syslab 平台发送消息 repl/getvariables
- Syslab 平台执行消息处理后，将结果发送给 APP SDK
- APP 获取到结果后，在图形界面将变量列表展示给用户

- 关闭 APP

- 用户关闭 APP 程序
- APP 删除 APP SDK 实例
- APP SDK 向 Syslab 平台发送 APP 关闭的通知，Syslab 关闭管道连接，同时关闭其他通信 IO 资源

1.2.2 开发运行流程

APP 的开发运行流程包括以下过程：

- APP 开发：基于 APP 应用开发环境，编写代码开发 GUI 图形用户界面，开发具体的业务逻辑，实现与科学计算环境的数据交互，调用科学计算环境的函数和算法。
- APP 测试：APP 开发完成后的测试验证工作，包括开发者自测试和专业测试。本文侧重于开发者的自测试，包括 2 个测试场景，场景 1 为不依赖于科学计算环境，通过打桩测试实现 APP 的独立测试，验证 APP 自身的功能，场景 2 为与科学计算环境的集成联调测试。
- APP 打包：APP 的代码开发完成后，将 APP 打包成独立可运行的程序（例如 exe 或 sh 脚本），对于依赖的动态链接库等文件也要一起打包。
- APP 安装：指 APP 打包好后，将 APP 安装和集成到科学计算环境中，实现 APP 的可查询、可运行、可管理。
- APP 使用：APP 安装成功后，可以在科学计算环境中使用统一的用法来使用 APP，包括查询 APP、启动 APP、使用 APP 等。

APP 的开发运行流程如下图所示。

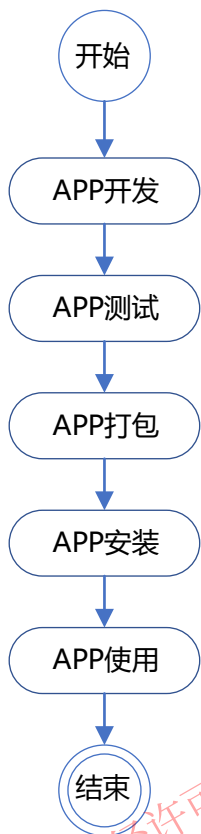


图 1-3 APP 的开发运行流程图

1.2.2.1 APP 开发

用户可以使用多种主流的图形应用开发平台（PyQt、C++/Qt、JavaScript 等）来开发 APP。具体过程包括编写代码开发 GUI 图形用户界面，开发具体的业务逻辑，实现与科学计算环境的数据交互，调用科学计算环境的函数和算法。APP 的开发过程如下所示。

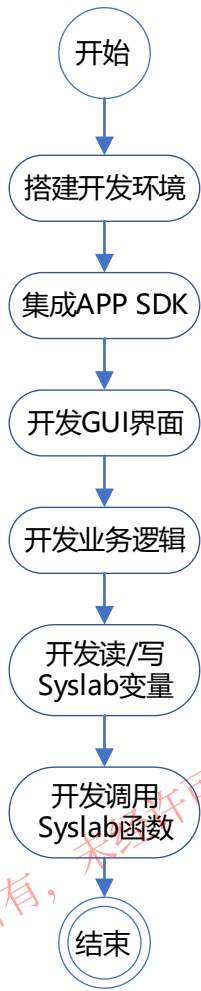


图 1-4 APP 开发流程图

1.2.2.2 APP 测试

APP 开发完成后的测试验证工作，包括开发者自测试和专业测试。本文侧重于开发者的自测试，包括 2 个测试场景，场景 1 为不依赖于科学计算环境，通过打桩测试实现 APP 的独立测试，验证 APP 自身的功能，场景 2 为与科学计算环境的集成联调测试。具体流程图如下所示。

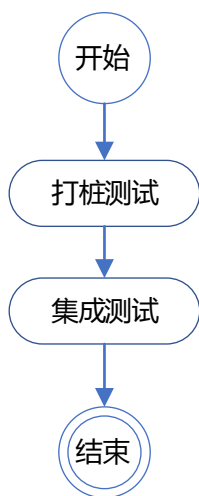


图 1-5 APP 测试流程图

1.2.2.3 APP 打包

APP 打包遵循具体 APP 开发环境的要求,打包好的 APP 程序需独立可运行,不需再另外安装软件或执行其他的操作。

1.2.2.4 APP 安装

APP 打包好后,将 APP 安装和集成到科学计算环境中,实现 APP 的可查询、可运行、可管理。APP 安装和卸载都是在 Syslab 中操作,APP 安装成功后才能在 Syslab 中使用。具体流程图如下所示。

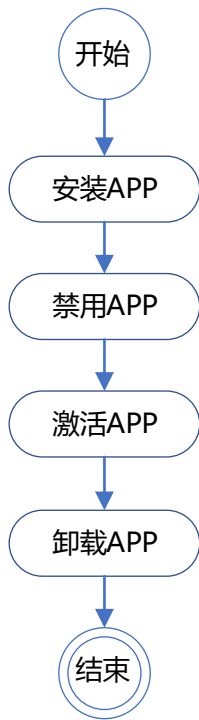


图 1-6 APP 安装流程图

1.2.2.5 APP 使用

APP 安装成功后,可以在 Syslab 平台中查询到 APP 信息,用户可以在 Syslab 中启动并使用 APP。具体流程图如下所示。

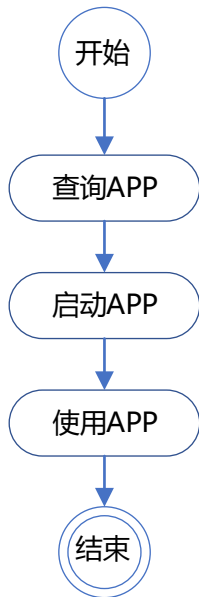


图 1-7 APP 使用流程图

2. 如何快速开发 APP

2.1 曲线拟合 APP 介绍

下面以 Syslab 平台自带的 APP“曲线拟合工具”为例，讲述开发和使用 APP 的方法。曲线拟合 APP 提供了一个灵活的界面，您可以在其中使用拟合算法来交互拟合数据并查看曲线图。

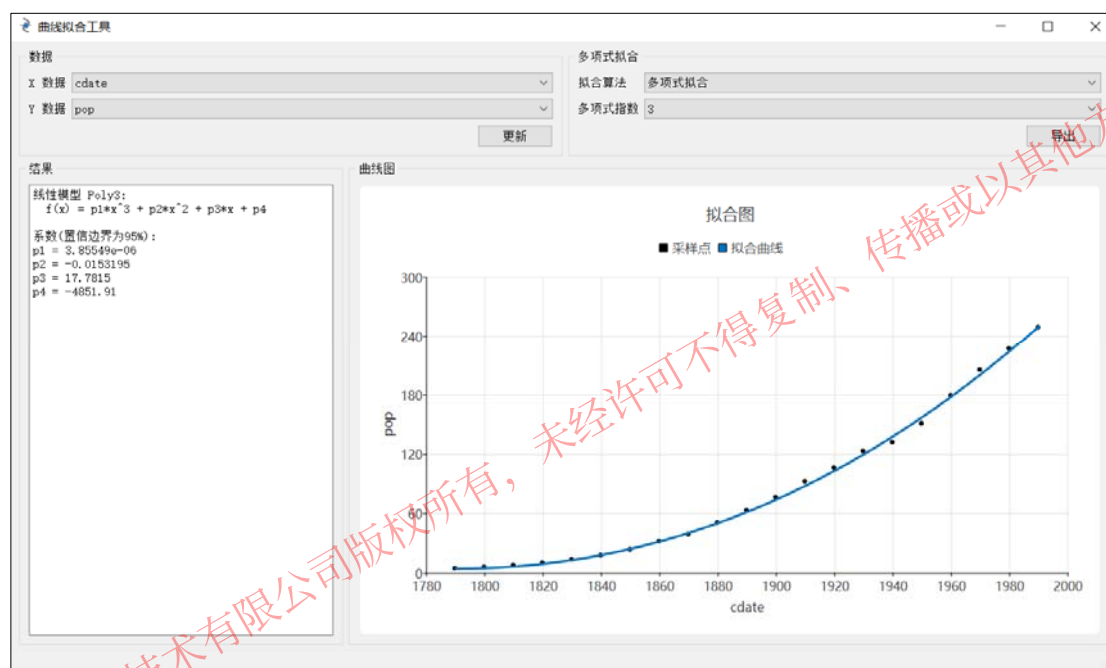


图 2-1 曲线拟合工具

2.2 APP 开发测试打包

如果您要自己开发 APP，请参考[第 3 章](#)。

2.3 APP 安装使用

Syslab 平台提供可视化管理和命令式管理两种方式，都可以实现对 APP 的安装和使用。

2.3.1 可视化管理

Syslab 提供 APP 可视化管理功能，支持安装、启动、编辑、卸载 APP 等功能，APP 管理入口位于工具栏 Ribbon 的 APP 菜单页。

2.3.1.1 APP 管理入口

Syslab 的 APP 管理入口，位于工具栏 Ribbon 的 APP 菜单页，提供 APP 管理的快捷操作。位置如下图：



图 2-2 APP 管理入口

APP 的 Ribbon 面板主要由两部分组成：

- 安装 APP：APP 安装入口。点击按钮，弹出安装 APP 对话框，填写 APP 的相关配置项，点击确定，即可安装 APP 到 APP 展示区。
- APP 展示区：APP 启动入口。展示 Syslab 内置 APP 和用户自定义安装的 APP，点击 APP 图标，即可启动 APP。点击最右侧的下拉箭头，弹出下拉面板，显示所有已安装的 APP。

2.3.1.2 安装 APP

Syslab APP 管理提供安装 APP 功能，点击安装 APP 按钮，弹出安装 APP 对话框，填写 APP 的相关配置项，点击确定，即可安装 APP 到 APP 展示区。操作步骤如下图所示：

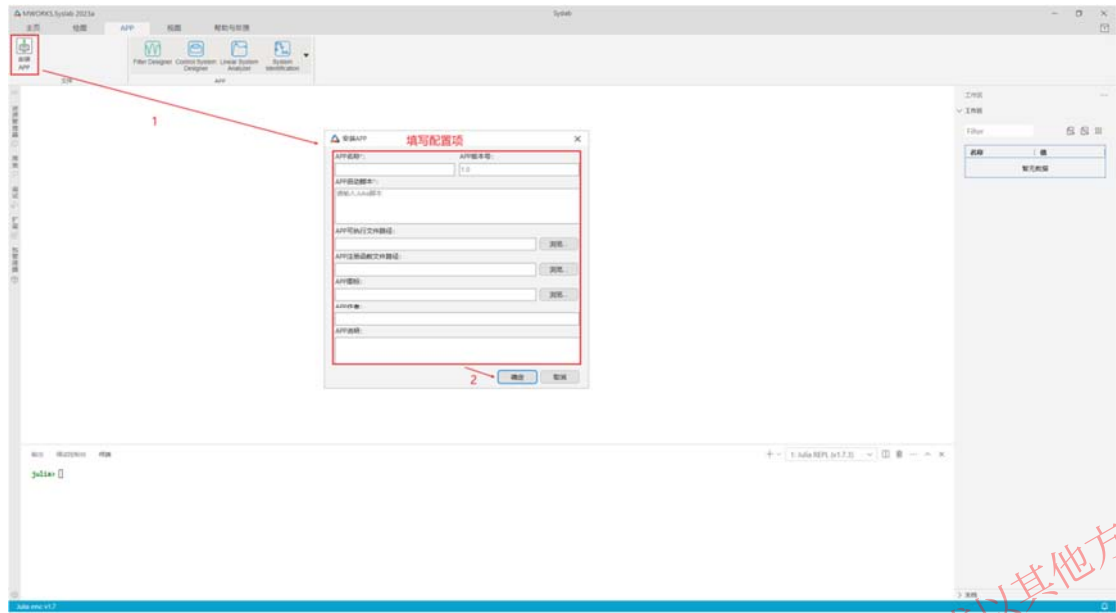


图 2-3 安装 APP

本节以曲线拟合工具 APP 为例，介绍如何填写安装 APP 的相关配置项。

Syslab 中提供了曲线拟合工具 APP 示例，位于<Syslab 安装路径>/Examples/10 AppDemos 下，建议用户将 APP 放到统一的目录下进行管理。Syslab 提供了用户 APP 存放目录，windows 系统位于 C:/Users/Public/TongYuan/syslab-julia/Apps 下；linux 系统位于~/TongYuan/syslab-julia/Apps 下。

本节以 windows 系统为例，在用户 APP 目录下创建 Cftool 文件夹，并将示例程序<Syslab 安装路径>/Examples/10 AppDemos/cpp/dist/cftool-1.0-win64.zip 解压后，将内容拷贝到该文件夹下。

OS (C:) > 用户 > 公用 > TongYuan > syslab-julia > Apps > Cftool >				
	名称	修改日期	类型	大小
	iconengines	2023-5-30 19:19	文件夹	
	imageformats	2023-5-30 19:19	文件夹	
	platforms	2023-5-30 19:19	文件夹	
	styles	2023-5-30 19:19	文件夹	
	translations	2023-5-30 19:19	文件夹	
	census.jl	2023-4-17 17:23	JL 文件	1 KB
	cftool.jl	2023-4-17 17:23	JL 文件	1 KB
	CurveFitTool.exe	2023-5-30 17:17	应用程序	258 KB
	CurveFitTool.pdb	2023-5-30 17:17	Program Debug ...	3,868 KB
	D3Dcompiler_47.dll	2014-3-11 18:54	应用程序扩展	4,077 KB
	libEGL.dll	2020-3-27 21:18	应用程序扩展	24 KB
	libGLSv2.dll	2020-3-27 21:18	应用程序扩展	3,491 KB
	opengl32sw.dll	2016-6-14 20:00	应用程序扩展	20,433 KB
	Qt5Charts.dll	2020-3-28 4:46	应用程序扩展	1,401 KB
	Qt5Core.dll	2020-3-27 21:18	应用程序扩展	6,049 KB
	Qt5Gui.dll	2020-3-27 21:18	应用程序扩展	6,963 KB
	Qt5Svg.dll	2020-3-28 2:41	应用程序扩展	380 KB
	Qt5Widgets.dll	2020-3-27 21:18	应用程序扩展	5,466 KB
	SyslabAppSdk.lib	2023-5-30 17:17	Object File Library	6,093 KB
	vc_redist.x64.exe	2022-10-17 10:34	应用程序	14,953 KB

图 2-4 曲线拟合 APP 文件路径

OS (C:) > 用户 > 公用 > TongYuan > syslab-julia > Apps				
	名称	修改日期	类型	大小
	Cftool	2023-5-5 10:04	文件夹	
	app_default.svg	2023-5-6 14:21	360 se HTML Do...	1 KB
	app_info.json	2023-5-20 9:26	JSON File	2 KB

图 2-5 用户 APP 目录

- Cftool: Cftool APP 文件夹(内部包含示例 APP)。
- app_default.svg: 默认 APP 图标（系统提供，用户无需修改）。
- app_info.json: 用户安装 APP 的配置文件(系统提供,用户无需修改)。

A. 操作步骤

1. 打开 Syslab，Ribbon 工具栏切换至 APP 菜单页。



图 2-6 APP Tab 页

2. 点击安装 APP 按钮,弹出安装 APP 对话框,如图所示,编辑相关配置项。

图 2-7 安装 APP 编辑界面

- APP 名称（必填）：当前安装 APP 的名称。
- APP 版本号：当前安装 APP 的版本号（缺省则默认 APP 版本号为 1.0）。
- APP 启动脚本（必填）：当前 APP 的启动脚本，点击 APP 时，将在 Julia REPL 终端执行此脚本。本例中设置启动脚本为 `SyslabApp.start("Cftool",2)`，其中：
 - `SyslabApp.start`：内置提供的启动 APP 命令；
 - “Cftool”：APP 的名称；
 - “2”：APP 的输入参数。针对本示例，表示 APP 启动时设置多项式指数为 2。
- APP 可执行文件路径：当前 APP 的可执行文件的路径。
- APP 注册函数文件路径：高级选项，启动 APP 时将执行该注册函数

文件(*.jl), 详细用法参见控制系统设计 APP 的相关配置。

- APP 图标: 当前 APP 的图标, 用户可以选择本地路径中 svg、jpeg、jpg、png 格式图片, 若不进行设置, 则统一使用默认图标。
- APP 作者: 当前 APP 的作者。
- APP 说明: 当前 APP 的说明。

3. 安装验证: 点击确定按钮, APP 安装成功, Ribbon 面板成功添加 Cftool APP。



图 2-8 APP 安装完成

2.3.1.3 启动 APP

点击已安装的 Cftool APP, REPL 终端执行启动脚本, 弹出曲线拟合工具窗口, 启动 APP 成功。

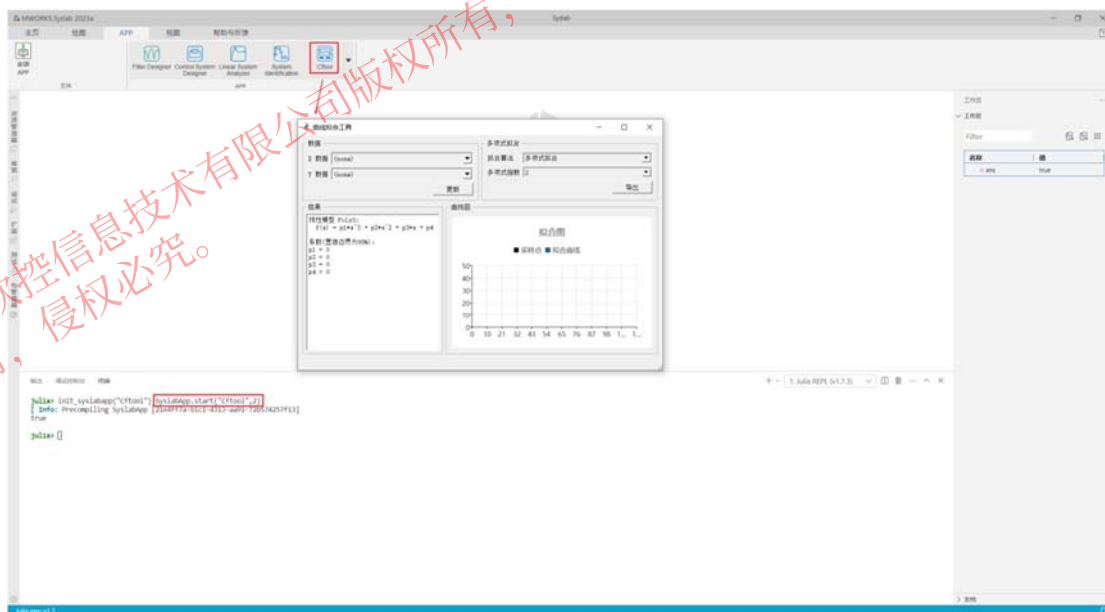


图 2-9 启动 APP

2.3.1.4 编辑 APP

Syslab APP 管理提供编辑 APP 功能, 支持对已安装 APP 的相关信息进行修改。点击 APP 面板最右侧的下拉箭头, 弹出下拉面板, 显示所有已安装的 APP,

右键已安装的 APP，点击编辑按钮，弹出编辑 APP 对话框，支持对 APP 名称以外的 APP 信息进行修改，如修改 APP 版本号为 2.0。

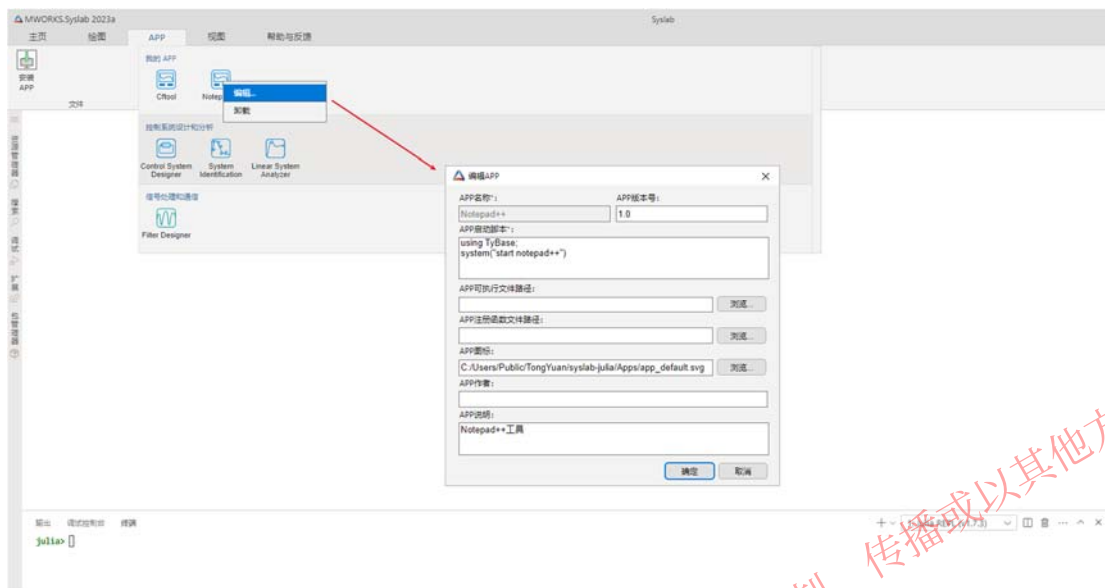


图 2-10 编辑 APP

点击确定按钮，保存编辑内容，Ribbon 面板同步刷新，当鼠标悬停在 Notepad++APP 时，显示版本号更新为 2.0。



图 2-11 APP 信息

2.3.1.5 卸载 APP

Syslab APP 管理提供卸载 APP 功能，点击 APP 面板最右侧的下拉箭头，弹出下拉菜单，显示所有已安装的 APP，右键已安装的 APP，点击卸载按钮，选中的 APP 从面板移除。

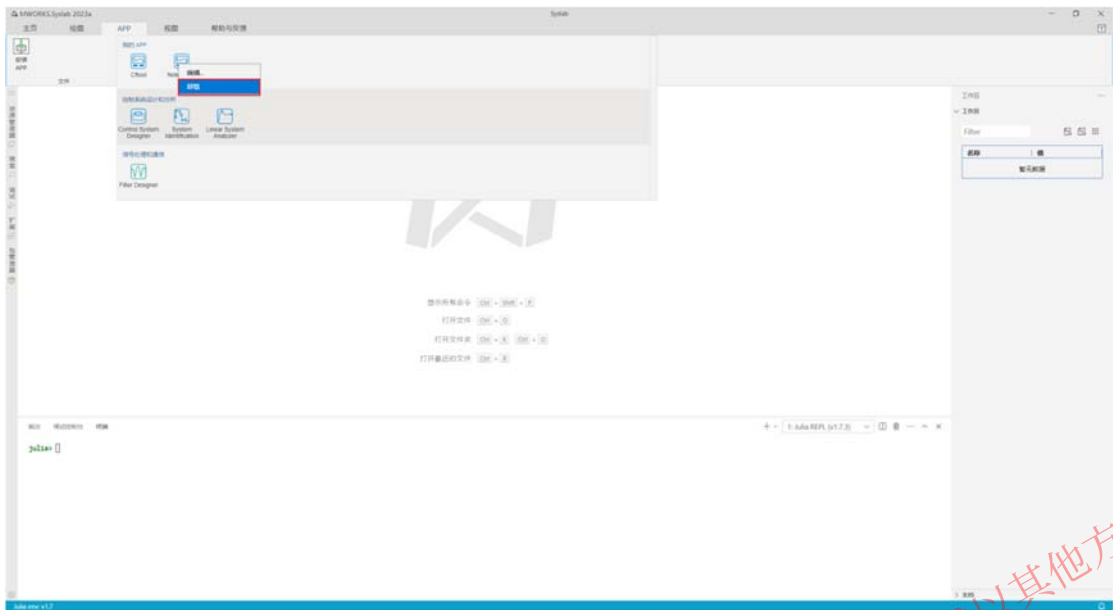


图 2-12 删除 APP

2.3.1.6 APP 与 Syslab 交互

2.3.1.6.1 在 Syslab 中导入测试数据

“曲线拟合工具”示例 APP 提供了测试数据，位于 <Syslab 安装路径>\Examples\10 AppDemos\CurveFitTool\test\census.jl。回到 Syslab 命令行窗口，执行导入数据的命令，示例如下：

```
julia> include(raw"<Syslab 安 装 路 径 >\Examples\10
AppDemos\cpp\CurveFitTool\test\census.jl")
21-element Vector{Float64}:
 3.9
 5.3
 7.2
 9.6
12.9
17.1
 ⋮
226.5
248.7
```

成功导入数据后，在工作区窗口中可以查看到变量的名称 cdate 和 pop。如下图所示：

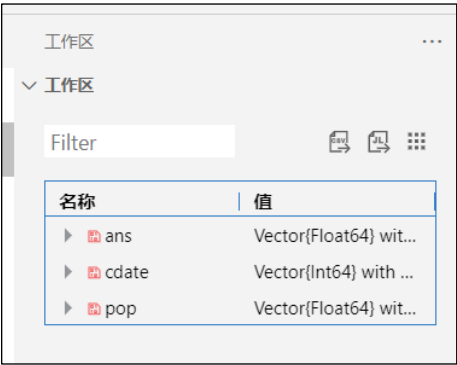


图 2-13 测试数据

cdate 为采样时间，它是一个数组，pop 为采样时间对应的数据，它也是一个数组。接下来的操作会使用到这 2 个数据。

2.3.1.6.2 从 Syslab 工作区获取数据

在“曲线拟合工具”的图形界面中，在“数据”区域点击“更新”按钮，如下图所示：

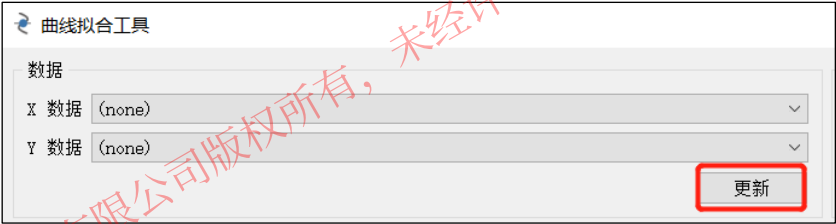


图 2-14 更新数据

更新完成后，在“X 数据”和“Y 数据”对应的下拉框中，分别选择数据 cdate 和 pop，线图控件将动态绘制拟合曲线，如下图所示：

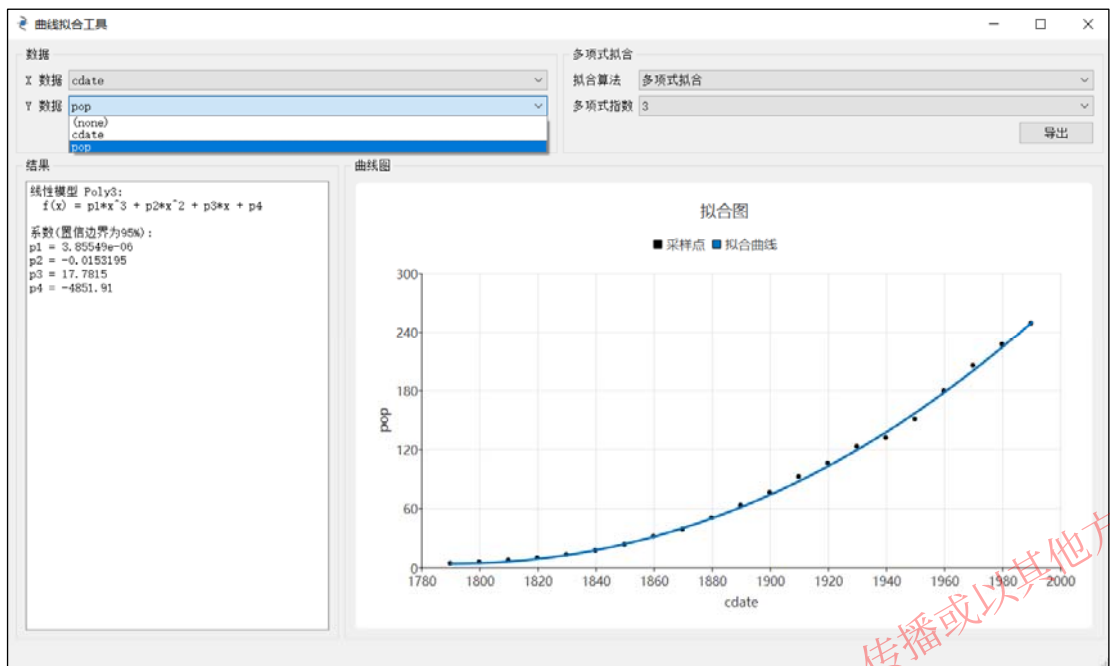


图 2-15 动态绘制曲线

您可以在“多项式拟合”区域，选择修改多项式指数，线图控件将动态绘制对应的新拟合曲线。

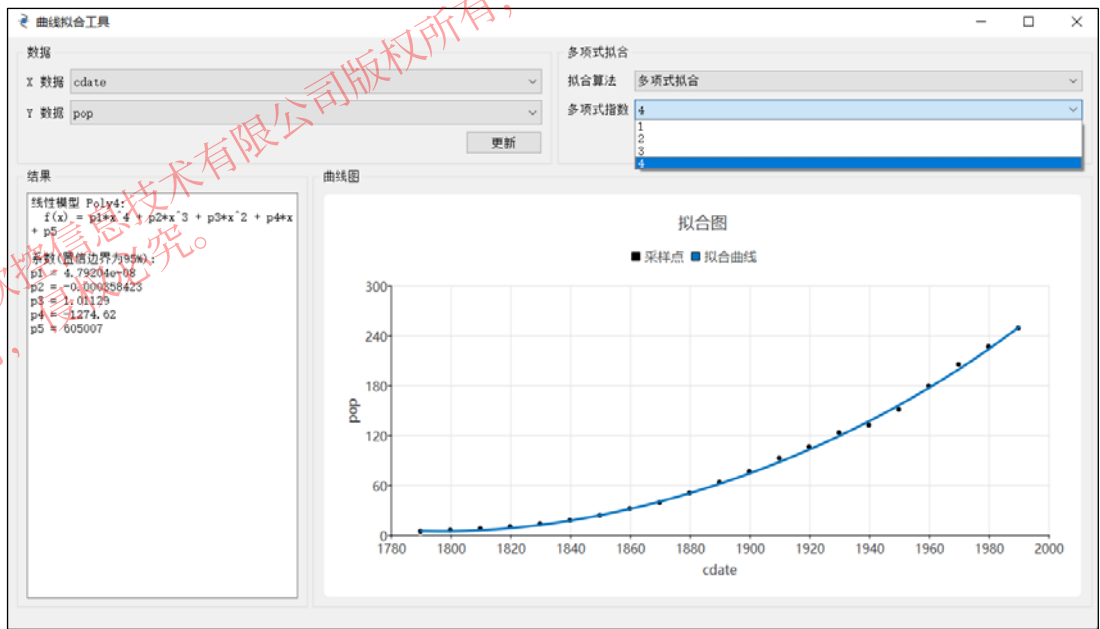


图 2-16 修改多项式指数

2.3.1.6.3 写数据到 Syslab 工作区

多项式拟合计算的结果包括拟合曲线的 y 值和多项式系数（p1、p2、p3...），

可以将这些结果写入 Syslab 工作区。在“多项式拟合”区域点击“导出”按钮，将弹出导出对话框，如下图所示：

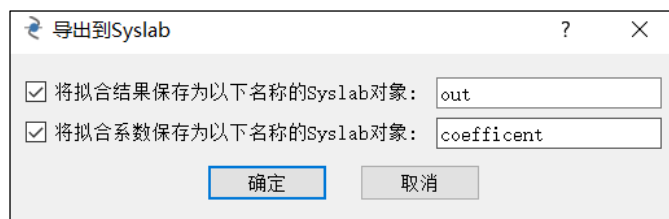


图 2-17 导出数据

点击“确定”按钮，导出完成后，在 Syslab 平台的工作区中将展示上述导出的数据，如下图所示：

苏州同元软控信息技术有限公司版权所有，未经许可不得复制、传播或以其他方式使用；
版权所有，侵权必究。

工作区

Filter

名称	值
ans	NamedTuple{(:p1, :p2, :p3, :p4), NTuple{4, Float64}}
cdate	Vector{Int64} with 21 elements
coefficient	NamedTuple{(:p1, :p2, :p3, :p4), NTuple{4, Float64}}
p1	3.85549e-6
p2	-0.0153195
p3	17.7815
p4	-4851.91
out	Vector{Float64} with 21 elements
1 ... 20	
1	4.26443
2	4.7847
3	6.405
4	9.14846
5	13.0382
6	18.0974
7	24.3492
8	31.8166
9	40.5229
10	50.4911
11	61.7444
12	74.306
13	88.1989
14	103.446
15	120.071
16	138.097
17	157.547
18	178.443
19	200.81
20	224.67
21 ... 21	
pop	Vector{Float64} with 21 elements
start_time	0x00009028c520b358

图 2-18 工作区导出的数据

您也可以在 Syslab 命令行窗口中，执行 `plot()` 命令来绘制结果曲线图，示例如下：

```
julia> plot(cdate, pop, ".", cdate, out)
2-element Vector{PyObject}:
PyObject <matplotlib.lines.Line2D object at 0x00000000D1B966D0>
PyObject <matplotlib.lines.Line2D object at 0x00000000D1B96A00>
```

Syslab 平台中绘制的结果曲线如下图所示：

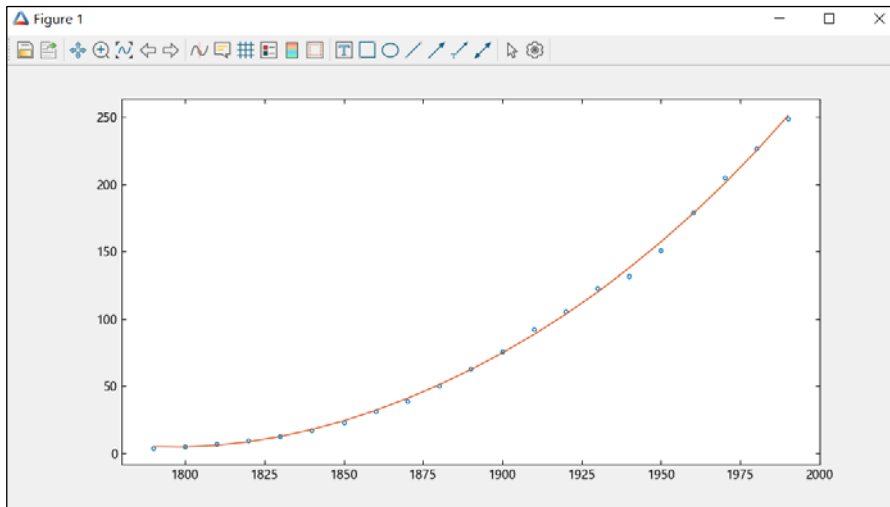


图 2-19 Syslab 中绘制的结果图

2.3.2 命令式管理

2.3.2.1 初始化

每次启动 Syslab 命令行窗口后，在使用 APP 前，需要执行命令 `init_SyslabApp()`，以初始化 APP 管理的上下文环境。示例如下：

```
julia> init_syslabapp()
```

2.3.2.2 安装 APP

以 Syslab 自带的 APP 示例“曲线拟合工具”为例，将示例程序<Syslab 安装路径>/Examples/10 AppDemos/cpp/dist/cftool-1.0-win64.zip 解压到同级目录后，安装 APP 的 julia 代码如下：

```
julia> cftool_info = SyslabApp.AppInfo("cftool",  
    raw"$(SYSLAB_HOME)/Examples/10 AppDemos/cpp/dist/cftool-1.0-win64/CurveFitTool.exe",  
    raw"$(SYSLAB_HOME)/Examples/10 AppDemos/cpp/dist/cftool-1.0-win64/cftool.jl",  
    "1.0",  
    "Tongyuan",  
    "曲线拟合工具")  
  
julia> SyslabApp.install(cftool_info)
```



```
true
```

以上 AppInfo 为 APP 的描述信息，具体属性描述如下：

- 参数 1: APP 名称，是 APP 的主键信息，必填项；
- 参数 2: APP 可执行文件路径，必填项；
- 参数 3: APP 启动脚本文件，可选项；
- 参数 4: APP 版本，可选项；
- 参数 5: APP 作者，可选项；
- 参数 6: APP 描述信息，可选项。

其中，“参数 3” APP 启动脚本文件的写法如下：

```
# 建议函数名与 APP 名称一致，函数参数由用户决定。
function cftool(term)
    # do something

    # 启动 APP 及传入参数，该参数将作为 APP 启动参数
    SyslabApp.start("cftool", term)
end
```

2.3.2.3 卸载 APP

在 Syslab 平台的命令行窗口中，使用命令 `SyslabApp.uninstall(name)` 来卸载 APP，入参为安装 APP 时输入的参数 1: APP 名称。APP 被卸载后就无法被使用了。示例如下：

```
julia> SyslabApp.uninstall("cftool")
true

julia> SyslabApp.start("cftool")
└ Warning: cftool is not exist!
└ @ SyslabApp .\scripts\packages\SyslabApp\src\SyslabApp.jl:151
false

julia> SyslabApp.get_app("cftool")
└ Warning: cftool is not exist!
└ @ SyslabApp .\scripts\packages\SyslabApp\src\SyslabApp.jl:181
false
```

2.3.2.4 禁用 APP

在 Syslab 平台的命令行窗口中，使用命令 `SyslabApp.disable(name)` 来禁用指

定名称的 APP，入参为安装 APP 时输入的参数 1：APP 名称。APP 被禁用后无法再启动。示例如下：

```
julia> SyslabApp.disable("cftool")
true

julia> cftool(3)
└ Warning: cftool is disabled!
└ @ SyslabApp .\scripts\packages\SyslabApp\src\SyslabApp.jl:156
false
```

2.3.2.5 激活 APP

在 Syslab 平台的命令行窗口中，使用命令 `SyslabApp.enable(name)` 来激活被禁用的 APP，入参为安装 APP 时输入的参数 1：APP 名称。APP 被激活后就可以再次启动了。示例如下：

```
julia> SyslabApp.enable("cftool")
true

julia> cftool(3)
true
```

2.3.2.6 启动 APP

到这一步您的“曲线拟合工具”已经安装成功。可以使用命令 `SyslabApp.start("cftool")` 来启动该 APP。

命令启动 APP 的示例如下：

```
# 方法 1
julia> SyslabApp.start("cftool", 3)
true

# 方法 2：需要上一步的启动脚本文件
julia> cftool(3)
true
```

以上命令（二选一）执行成功后，将启动“曲线拟合工具”的图形界面。

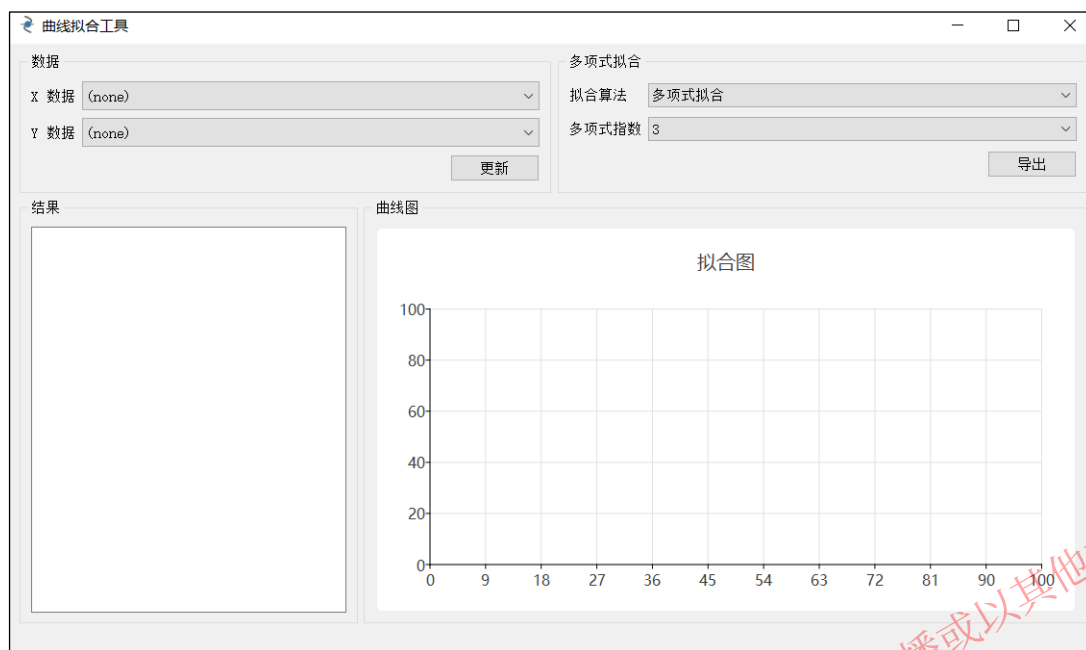


图 2-20 启动曲线拟合工具

2.3.2.7 查询 APP 列表

在 Syslab 平台的命令行窗口中，使用命令 `SyslabApp.get_apps()` 来查询注册到 Syslab 平台中的所有 APP 信息。示例如下：

```
julia> SyslabApp.get_apps()
Dict{String, Any} with 1 entry:
  "cftool" => Dict{String, Any}("registerFunc"=>"cftool", "name"=>"cftool",
    "author"=>"Tongyuan", "enabled"=>true, "version"=>"1.0",
    "path"=>"E:/MwSyslab/Test/Examples/10 AppDemos/x64/Debug/C...
```

2.3.2.8 查询指定 APP 信息

在 Syslab 平台的命令行窗口中，使用命令 `SyslabApp.get_app(name)` 来查询注册到 Syslab 平台中的指定名称的 APP 信息，入参为安装 APP 时输入的参数 1：APP 名称。示例如下：

```
julia> SyslabApp.get_app("cftool")
Dict{String, Any} with 7 entries:
  "registerFunc" => "cftool"
  "name"         => "cftool"
  "author"       => "Tongyuan"
  "enabled"      => true
  "version"      => "1.0"
```

```
"path"      => "./Test/Examples/10 AppDemos/x64/Debug/CurveFitTool.exe"
"description" => "曲线拟合工具箱"
```

3. 综合案例介绍

3.1 Python 版 APP

3.1.1 APP 开发

3.1.1.1 创建 APP 工程

3.1.1.1.1 准备好开发环境

表 3-1 软件开发环境

开发环境	版本号	说明
Syslab（推荐）	2023b	集成开发环境
Python	3.7.5	解释器
PyQt5	5.12.3	Python 版 Qt 包
pyqt5-tools	5.12.3	Qt 工具，如 Qt Designer、Liguist 等
PyQtChart	5.12.0	Python 版 QtChart 包
pyminstaller	5.1	将工程打包为可执行文件
PYQT Integration	V0.2.0	插件

A. Syslab

Python APP 开发建议使用 Syslab，也可以选择 PyCharm 等其他集成开发环境，软件安装方法请参考相关说明。

B. Python

Syslab2023a 内置了 Python3.7.5 环境，无需安装即可直接使用。

如果需要其他版本的 Python，请在 Python 官网进行下载安装，地址为：

<https://www.python.org/downloads/>

C. 依赖包安装

确保已经安装完成 Python 后，需要进行 APP 开发的依赖包安装。

(1). 打开命令行终端

打开 Syslab 软件，选择当前工作目录后，点击终端界面中右上角的图标“+”的下拉列表，新建 Command Prompt 终端。

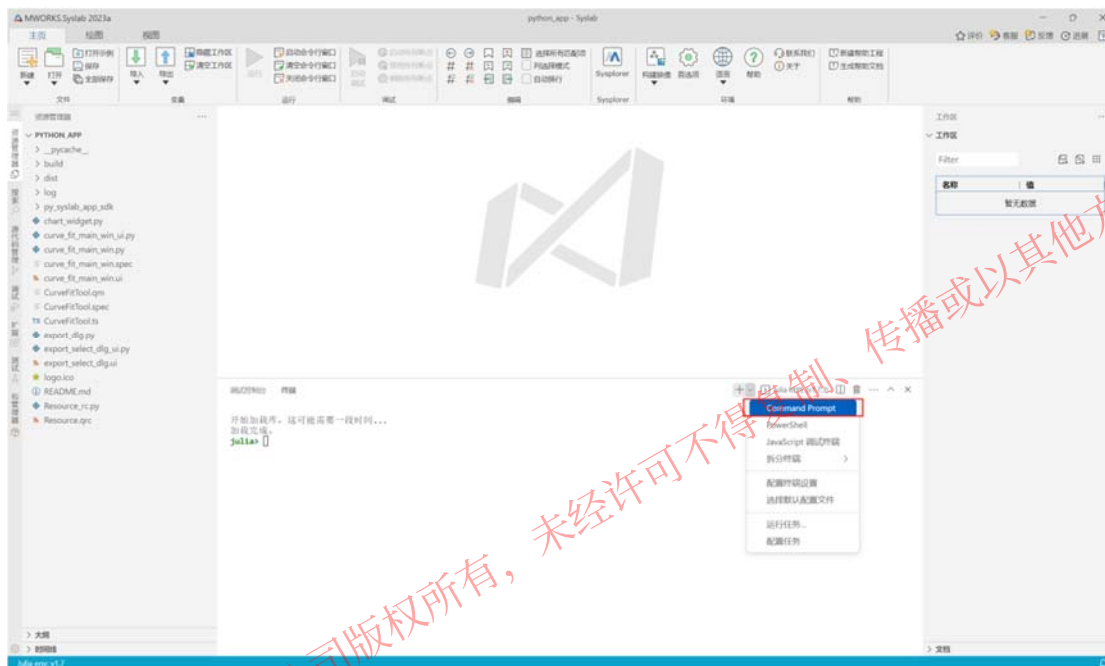


图 3-1 新建终端

(2). 检查依赖包安装情况

请输入以下指令查看内置的 Python 是否已默认安装依赖的 PyQt5、pyqt5-tools、PyQtChart、pyinstaller 第三方包。

```
>pip list
```

如果未安装相关包，请继续阅读本小节，否则跳转至下一小节“PYQT Integration 插件”。

(3). 安装 PyQt5

请注意 PyQt5、pyqt5-tools 和 PyQtChart 的版本需一致，以免出现不兼容的情况，这里采用推荐的 5.12.3 版本进行安装，建议使用国内镜像进行下载：

```
>pip3 install PyQt5==5.12.3 -i http://pypi.doubanio.com/simple/ --trusted-host pypi.doubanio.com
```

(4). 安装 pyqt5-tools

```
>pip3 install pyqt5-tools==5.12.3.1.6rc1 -i
```

```
http://pypi.doubanio.com/simple/ --trusted-host pypi.doubanio.com
```

(5). 安装 PyQtChart

```
>pip3 install PyQtChart==5.12.0 -i http://pypi.doubanio.com/simple/
--trusted-host pypi.doubanio.com
```

(6). 安装 pyinstaller

```
>pip3 install pyinstaller==5.1 -i http://pypi.doubanio.com/simple/ -
--trusted-host pypi.doubanio.com
```

D. PYQT Integration 插件

(1). 安装“PYQT Integration”插件

在 Syslab 界面中, 点击左侧侧边栏中的“扩展图标”(形如四方块的图标), 在顶部搜索框中输入“PYQT Integration”, 然后选择该插件, 点击安装。

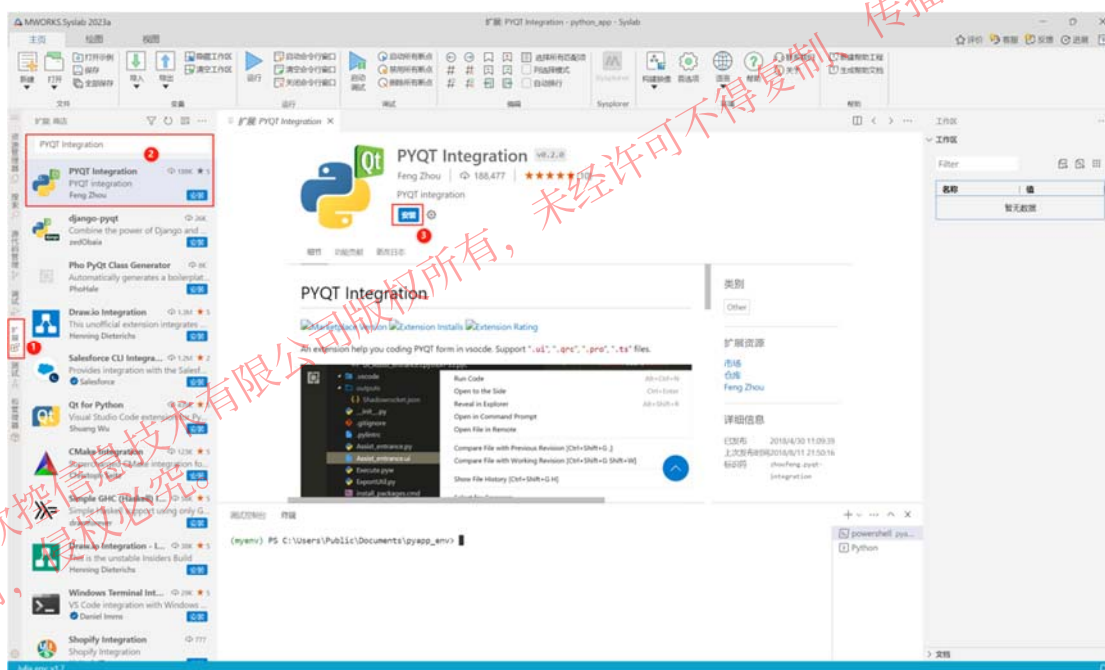


图 3-2 安装 PYQT Integration 插件

(2). 配置“PYQT Integration”插件

完成 PYQT Integration 插件安装后, 需要进行参数配置。在“扩展: PYQT Integration”界面中点击插件卡片下方的形如齿轮的“设置”图标按钮, 弹出选择菜单, 在菜单中选择“扩展设置”。

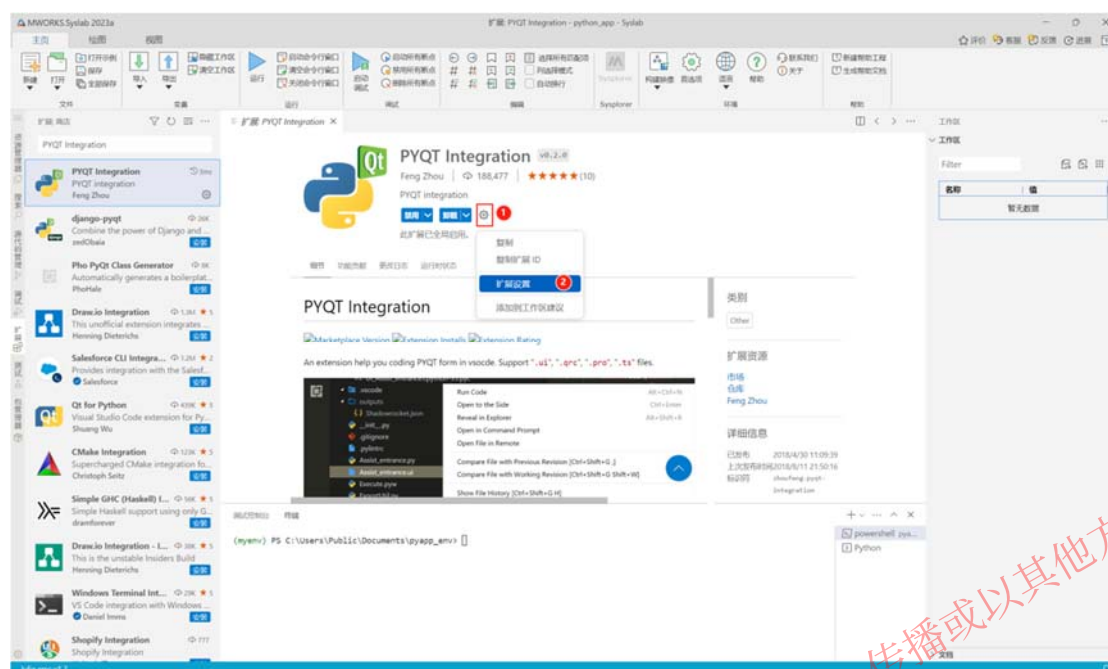


图 3-3 插件设置

在设置页面中插件需要配置三个路径。其中的“Python 解释器位置”为在此例中为“C:\Users\Public\TongYuan\julia\miniforge3\Scripts”。

- Pyrc:Cmd

Python 解释器位置\pyrc5

- Pyuic:Cmd

Python 解释器位置\pyuic5

- Qt designer:Path

Python 解释器位置\pyqt5designer

配置完成后的界面如下图所示。

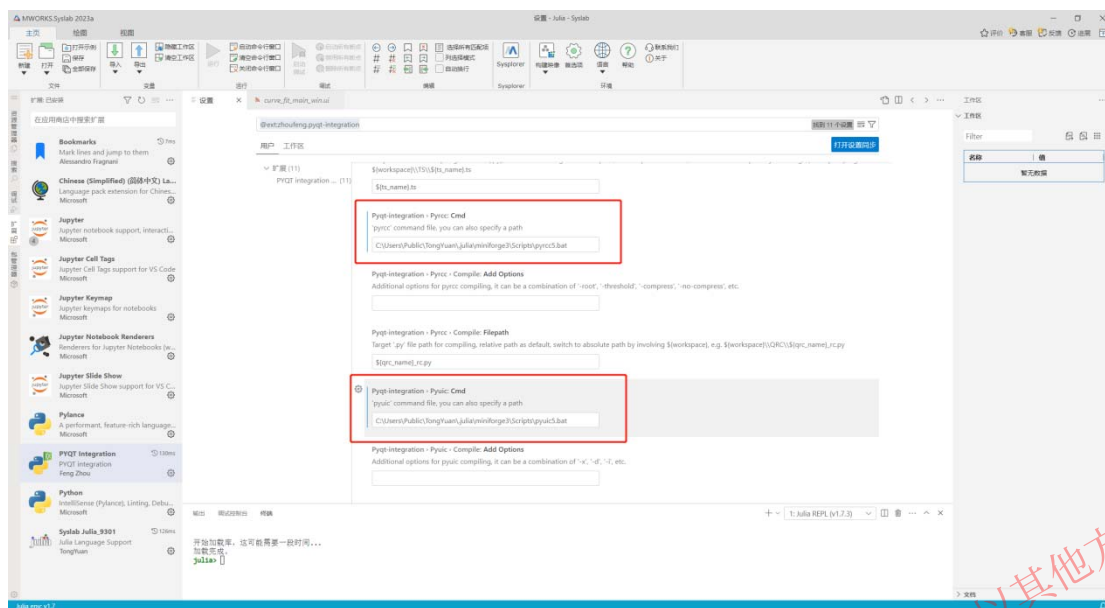


图 3-4 插件设置界面配置

3.1.1.1.2 新建 Qt 图形应用工程

A. 创建 Qt UI 文件

安装好插件后，在资源管理器中点击右键，选择“PYQT: New Form”打开 Qt Designer 软件。

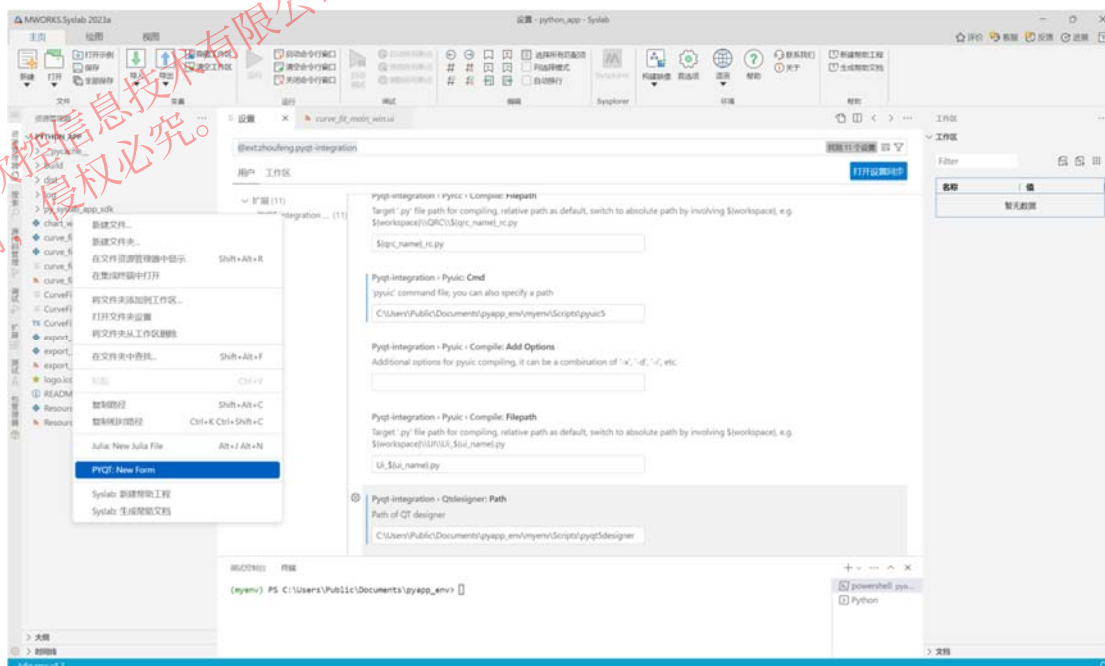


图 3-5 打开 Qt Designer

以 Python 版 CurveFitTool 为例，进一步开发 UI 界面，UI 界面的开发方法在此不做说明。开发好的 UI 界面效果如下图所示：

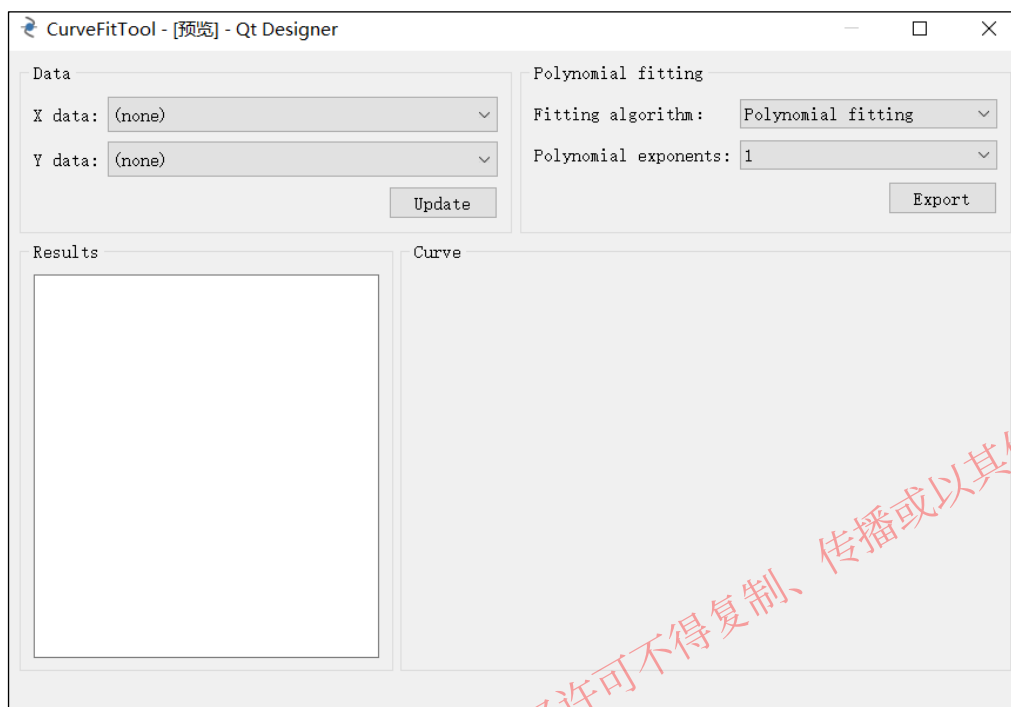


图 3-6 UI 界面设计

以上界面交互中，“Data”数据部分需要从 Syslab 工作空间获取数组类型的变量名列表，以及通过变量名获取变量值；“Export”导出部分需要将计算结果写入 Syslab 工作空间；“Results”输出拟合后的表达式；“Curve”区域绘制二维曲线图。

将此 UI 文件保存为 “curve_fit_main_win.ui”。

(1). 创建资源文件

在同一文件夹中新建 “Resource.qrc” 文件，输入以下内容：

```
<RCC>
  <qresource prefix="/CurveFitTool">
    <file>logo.ico</file>
  </qresource>
</RCC>
```

其中 logo.ico 为 APP 图标文件，需要放在同一文件夹下。

B. 编译 UI 和资源文件

在 Syslab 中创建完成 UI 和资源文件后，需要调用 pyuic 和 pyrcc 进行编译

生成.py 格式的文件。选中需要编译的资源文件，点击右键弹出右键菜单，选择菜单栏中的“PYQT: Compile Resource”，将自动生成名为“文件名_rc.py”的文件。生成后的命名格式可以在“PYQT Integration”中进行设置。

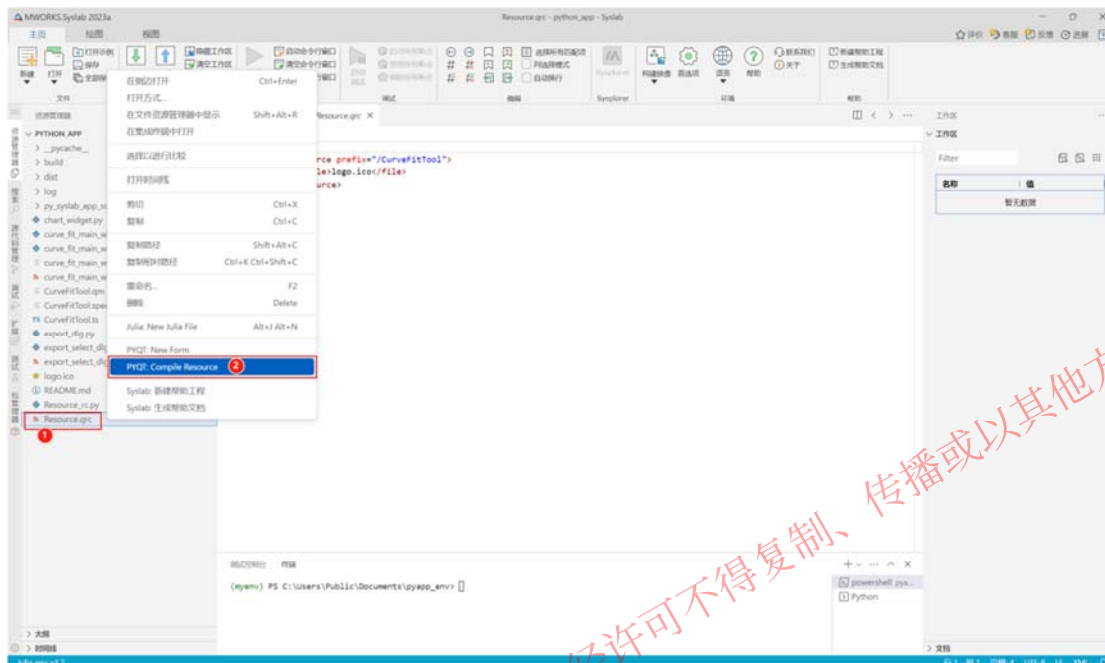


图 3-7 编译资源文件

同理，选中需要编译的 UI 文件，点击右键弹出右键菜单，选择菜单栏中的“PYQT: Compile From”，将自动生成名为“Ui_文件名.py”的文件。

C. 创建主窗口

使用 QMainWindow 类创建主窗口对象。

● curve_fit_main_win.py

```
from PyQt5.QtWidgets import QApplication, QMainWindow
from Ui_curve_fit_main_win import *
from Ui_export_select_dlg import *

class CurveFitMainWin(QMainWindow, Ui_CurveFitMainWin):
    def __init__(self, parent = None):
        super().__init__(parent)          # 调用父类构造函数，创建窗体
        self.ui = Ui_CurveFitMainWin()    # 创建 UI 对象
        self.ui.setupUi(self)             # 构造 UI 界面

    # ...
```

3.1.1.2 APP 开发与集成

3.1.1.2.1 使用 Python AppSdk

SyslabAPPSdk 是负责与 Syslab 平台通信交互的 SDK, SyslabAPPSdk 可以在路径: <Syslab 安装路径>\Examples\10 APPDemos\python\py_syslab_app_sdk 下找到, 将 py_syslab_app_sdk 文件夹拷贝到 Qt 工程目录下。

完成 py_syslab_app_sdk 包的拷贝后, 导入 SyslabAPPSdk 包, 就可在工程中使用相关接口。导入 SyslabAPPSdk 包的示例代码如下:

```
from py_syslab_app_sdk.syslab_app_sdk import SyslabAppSdk
from py_syslab_app_sdk.variable_info import VariableInfo
```

3.1.1.2.2 传递 APP 启动参数

Syslab 平台启动 APP 程序时, 会传递给 APP 进程以下启动参数:

- argv[0]: 用于调用程序的命令;
- argv[1]: 管道名, 由 Syslab 平台生成后传递给 APP;
- argv[2~3]: Julia 相关路径参数, 由 Syslab 平台生成后传递给 APP;
- argv[4~]: 由用户在 APP 启动函数中输入。

以 Python 版 CurveFitTool 为例, 在文件 “curve_fit_main_win.py” 中的 “__main__” 启动函数中负责解析这些参数, 其中管道名是必须解析的, Julia 参数和其他参数 APP 可以按需解析和使用。示例代码如下:

```
# 与 Syslab 平台集成后, 自动获取管道名
if len(sys.argv) > 1 and "--pipe=" in sys.argv[1]:
    pipe_name = sys.argv[1].replace("--pipe=", "")
    main_window.set_pipe_name(pipe_name)

# 用户自定义启动参数举例: 获取多项式系数
if len(sys.argv) > 4:
    qstr_term = str(sys.argv[4])
    main_window.set_term(qstr_term)
```

以管道名参数为例, 解析后的管道名要传递给 QMainWindow 的实现类 CurveFitMainWin, 用以创建命名管道客户端。在 CurveFitMainWin 类中新增函数 set_pipe_name(), 示例代码如下:

● `curve_fit_main_win.py`

```
def set_pipe_name(self, pipe_name):
    if self.m_syslab_sdk == None:
        self.m_syslab_sdk = SyslabAppSdk("CurveFitTool", pipe_name)
        self.slot_update_syslab_var()
```

其他参数以类似的方法可以传递给 `CurveFitMainWin`，在此不再赘述。

`SyslabAPPSdk` 主要提供以下接口：

- 获取变量列表
- 查询变量值
- 发送脚本到 `Syslab` 执行
- 关闭命名管道

下文将讲述这些接口的具体使用方法。

3.1.1.2.3 获取变量列表

```
def mw_get_variables(show_modules: bool) -> (list | Literal[False])
```

功能	获取 Syslab 工作区变量列表
参数	show_modules: 是否显示模块列表，一般为 False
返回值	False-失败 变量列表-成功

`CurveFitTool` 中调用该接口的示例代码如下：

● `curve_fit_main_win.py`

```
variables = []
if self.m_syslab_sdk:
    variables = self.m_syslab_sdk.mw_get_variables(False)

for var in variables:
    if r"Vector{Int64}" in var.get_type() or r"Vector{Float64}" in var.get_type():
        self.ui.comboBox_x.addItem(var.get_name())
        self.ui.comboBox_y.addItem(var.get_name())
```

3.1.1.2.4 获取变量值

```
def mw_get_value(var_name: str) -> (str | Literal[False])
```

功能	获取工作区变量值
----	----------

参数	var: 变量名, 可以为子变量 a.b
返回值	变量值字符串-成功 False-失败

CurveFitTool 中调用该接口的示例代码如下:

```
value = self.m_syslab_sdk.mw_get_value(str(name))
str_value = str(value)
var_lst = str_value[1:-1].split(",")
datas = []
for var_str in var_lst:
    datas.append(float(var_str))
```

3.1.1.2.5 运行脚本

```
def mw_run_script(code: str,
                  show_code_in_repl: bool,
                  show_result_in_repl: bool) -> (str | Literal[False])
```

功能	在 syslab 工作区执行 Julia 脚本代码
参数	code: 要运行的 julia 脚本 show_code_in_repl: 是否在 Syslab REPL 中显示代码 show_result_in_repl: 是否在 Syslab REPL 中显示结果
返回值	code 运行后的结果-成功 False-失败

CurveFitTool 中调用该接口的示例代码如下:

● curve_fit_main_win.py

```
if dlg.is_export_res():
    str_res_name = dlg.get_res_name()
    str_res_data = str(self.m_res_datas)
    str_script = f"{str_res_name}={str_res_data}"
    self.m_syslab_sdk.mw_run_script(str(str_script), None, None)
```

调用 Syslab 的数学 API 进行曲线拟合计算的示例代码如下, 第一步计算曲线拟合参数:

```
str_script = f"import TyMathCore;
temp_coeffi,temp_s = TyMathCore.polyfit(cdate, pop, {int_n})"
self.m_syslab_sdk.mw_run_script(str_script, False, False)
str_coeffi = self.m_syslab_sdk.mw_get_value("temp_coeffi")
```

第二步计算曲线拟合结果:

```

str_script      =      f"import      TyMathCore;      temp_results      =
TyMathCore.polyval(temp_coeffi, cdate)"
self.m_syslab_sdk.mw_run_script(str_script, False, False)
str_res = self.m_syslab_sdk.mw_get_value("temp_results")

```

3.1.1.2.6 关闭命名管道

```
def close_pipe() -> None
```

功能	发送请求，关闭命名管道，在关闭程序前调用
参数	无
返回值	无

在关闭 APP 前，需要调用一次上述关闭命名管道接口，与 Syslab 平台断开连接。CurveFitTool 中调用该接口的示例代码如下：

- curve_fit_main_win.py

```

def closeEvent(self, event):
    if self.m_syslab_sdk:
        self.m_syslab_sdk.close_pipe()
    event.accept()

```

到此为止，曲线拟合工具开发过程中要调用 SDK 的接口都已经讲完了，剩下的就是具体的业务逻辑代码开发。

该示例 Demo 的源代码，可以在 Syslab 安装软件以下路径获取到：<Syslab 安装路径>\Examples\10 AppDemos\python\。

3.1.2 APP 测试

3.1.2.1 打桩测试

以曲线拟合 APP 为例，该 APP 在运行时从 Syslab 平台获取数据后，进行曲线拟合计算，对 Syslab 平台有依赖。测试时为了解除对 Syslab 平台的依赖，可以通过打桩 mock 的方式，构造打桩数据，独立验证 APP 自身的功能。

打桩测试部分代码如下：

- curve_fit_main_win.py

```
global_is_mock = True
```

```

def slot_update_syslab_var(self):
    variables = []
    if global_is_mock:
        var1 = VariableInfo("cdata", "Vector{Float64}")
        var2 = VariableInfo("pop", "Vector{Float64}")
        variables.append(var1)
        variables.append(var2)
    elif self.m_syslab_sdk:
        variables = self.m_syslab_sdk.mw_get_variables(False)
        # 其他代码...

def get_mock_datas(self, var_name):
    if var_name == "cdata":
        return "[1790,1800,1810,1820,1830,1840,1850,1860," \
                "1870,1880,1890,1900,1910,1920,1930,1940," \
                "1950,1960,1970,1980,1990]"
    elif var_name == "pop":
        return "[3.9,5.3,7.2,9.6,12.9,17.1,23.1,31.4,38.6," \
                "50.2,62.9,76.0,92.0,105.7,122.8,131.7," \
                "150.7,179.0,205.0,226.5,248.7]"
    else:
        return ""

# 代码省略...

```

3.1.2.2 与 Syslab 平台集成调试

示例中的管道名需要在与 Syslab 平台集成后，才能在启动 APP 时自动获取。如果您要在开发过程中对 APP 进行调试，请按照以下步骤手动设置命名管道地址。

(1). 启动管道服务端

在 Syslab 中的 Julia REPL 终端中运行以下指令启动管道服务（新建 Julia REPL 终端时自动开启）：

```

VSCodeServer.start_app_pipe_server()

true

```

(2). 获取 Syslab 平台命名管道地址

```
VSCodeServer.app_pipe_name
```

```
"\\\\\\\\.\\\\pipe\\\\app-pipe-09cba4da-d3c5-4430-a89b-b18fcce3278f"
```

(3). 在 APP 的 “__main__” 启动函数中添加临时代码，设置上一步得到的命名管道地址，并注释自动获取管道名相关代码

```
if __name__ == "__main__":
    app = QApplication(sys.argv)          # 创建 GUI 应用程序
    main_window = CurveFitMainWin()       # 创建主窗体
    main_window.show()                    # 显示主窗体

    # 与 Syslab 平台集成后，自动获取管道名
    # if len(sys.argv) > 1 and "--pipe=" in sys.argv[1]:
    #     pipe_name = sys.argv[1].replace("--pipe=", "")
    #     main_window.set_pipe_name(pipe_name)

    # 用户自定义启动参数举例：获取多项式系数
    # if len(sys.argv) > 4:
    #     qstr_term = str(sys.argv[4])
    #     main_window.set_term(qstr_term)

    # 与 Syslab 平台集成调试
    pipe_name = "\\\\\\\\.\\\\pipe\\\\app-pipe-09cba4da-d3c5-4430-a89b-b18fcce3278f"
    main_window.set_pipe_name(pipe_name)

    sys.exit(app.exec_())
```

(4). 点击 Syslab “主页” 菜单栏中的 “启动调试” 图标，进行 APP 调试

3.1.2.3 集成测试

APP 开发和打包完成后，参考[章节 2.3](#)的内容，完成 APP 的集成测试。

3.1.3 APP 打包

PyQt5 的打包工具选择为 `pyinstaller`，它会自动检测和收集应用程序所需的所有依赖项，并将它们打包到生成的可执行文件中。命令的语法如下：

```
pyinstaller [options] script [script ...] | specfile
```


具体的使用详情可参考此链接：<https://pyinstaller.org/en/latest/usage.html>
在终端中输入以下指令，完成软件打包。

```
pyinstaller -n CurveFitTool curve_fit_main_win.py
```

打包完成后，会在同级目录下生成一个“dist”文件夹，文件夹中包含有名为“CurveFitTool.exe”的 APP。此外，如果需要国际化，请将翻译后的.qm 文件拷贝在“CurveFitTool.exe”的同级目录中，最终打包完成的文件夹如图 3-8 所示。

名称	修改日期	类型	大小
log	2023/5/19 11:41	文件夹	
PyQt5	2023/5/19 11:41	文件夹	
_bz2.pyd	2023/5/19 9:38	Python Extensio...	80 KB
_ctypes.pyd	2023/5/19 9:38	Python Extensio...	124 KB
_decimal.pyd	2023/5/19 9:38	Python Extensio...	284 KB
_hashlib.pyd	2023/5/19 9:38	Python Extensio...	31 KB
_lzma.pyd	2023/5/19 9:38	Python Extensio...	231 KB
_multiprocessing.pyd	2023/5/19 9:38	Python Extensio...	22 KB
_queue.pyd	2023/5/19 9:38	Python Extensio...	20 KB
_socket.pyd	2023/5/19 9:38	Python Extensio...	68 KB
_ssl.pyd	2023/5/19 9:38	Python Extensio...	112 KB
pyexpat.pyd	2023/5/19 9:38	Python Extensio...	188 KB
select.pyd	2023/5/19 9:38	Python Extensio...	19 KB
unicodedata.pyd	2023/5/19 9:38	Python Extensio...	1,042 KB
CurveFitTool.qm	2023/5/19 11:21	QM 文件	2 KB
base_library.zip	2023/5/19 11:26	ZIP 压缩文件	999 KB
CurveFitTool.exe	2023/5/19 11:26	应用程序	1,211 KB
api-ms-win-core-console-l1-1-0.dll	2023/5/19 9:38	应用程序扩展	22 KB
api-ms-win-core-datetime-l1-1-0.dll	2023/5/19 9:38	应用程序扩展	22 KB

图 3-8 Python APP 打包完成目录

3.2 C++ 版 APP

3.2.1 APP 开发

3.2.1.1 创建 APP 工程

3.2.1.1.1 准备好开发环境

- Visual Studio 2017
- Qt 5.14.2
 - 需要配置环境变量：
QTDIR = <QT 安装路径>\Qt5.14.2\5.14.2\msvc2017_64
- Qt 插件：Qt Visual Studio Tool 2.9.0

3.2.1.1.2 新建 Qt 图形应用工程

在 Visual Studio 中，选择：新建项目→Qt→Qt Widgets Application，点击“确定”按钮，创建新项目 CurveFitTool。

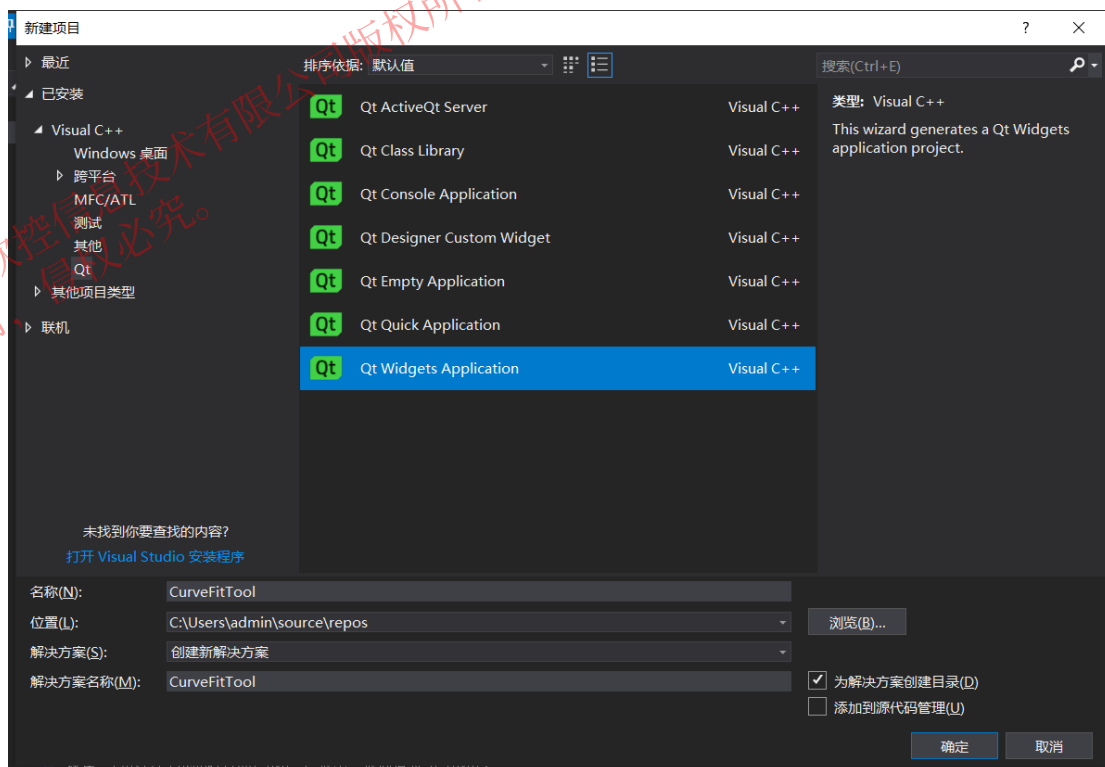


图 3-9 新建 VS 工程

3.2.1.2 APP 开发与集成

3.2.1.2.1 集成 C++ AppSdk

SyslabAppSdk 是负责与 Syslab 平台通信交互的 SDK，SyslabAppSdk 在路径 <Syslab 安装路径>\Examples\10 AppDemos\cpp\下可以找到。

(1) 将 SyslabAppSdk 拷贝到项目 CurveFitTool 所在的文件下，如下图所示：



图 3-10 拷贝 SDK 文件夹

(2) 在 Visual Studio 中将 SyslabAppSdk 工程添加到解决方案，选中解决方案后点击右键菜单：添加→现有项目，如下图所示：

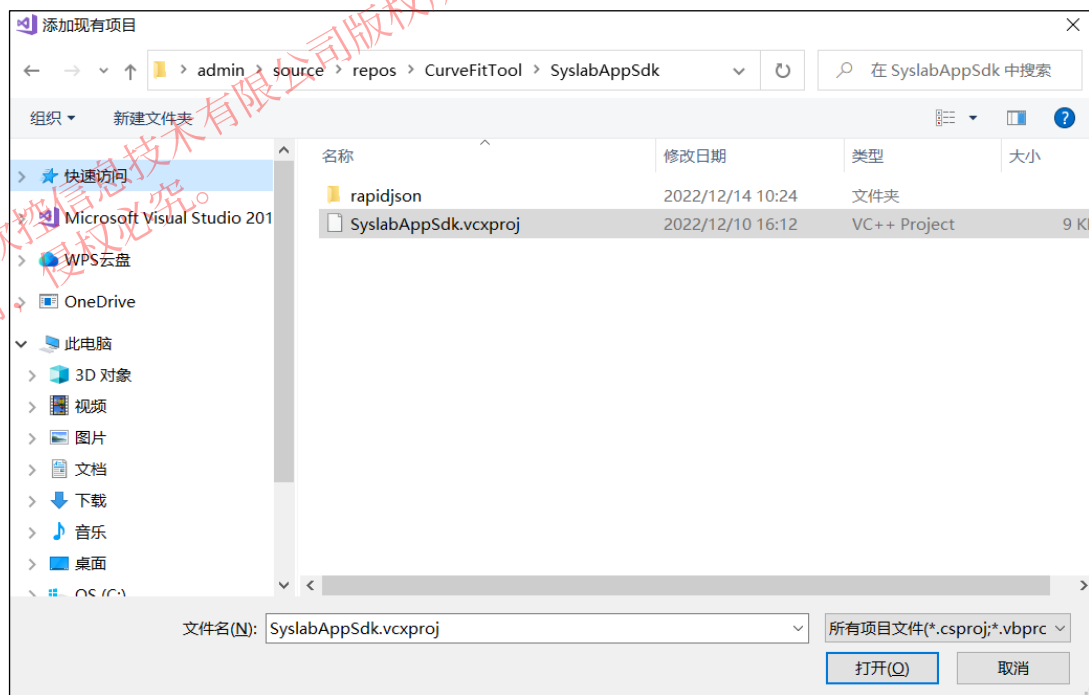


图 3-11 添加 SDK 项目

(3) 在 CurveFitTool 的项目属性中添加对 SyslabAppSdk 的附加包含目录依

赖：属性→C/C++→常规→附加包含目录，添加：..\SyslabAppSdk，如下图所示：

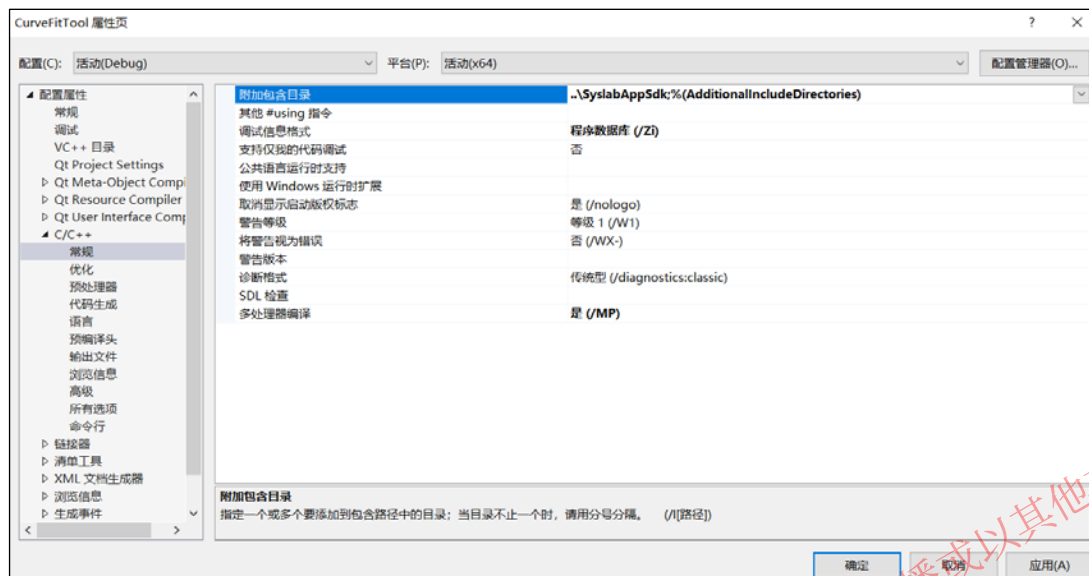


图 3-12 附加包含目录

(4) 在 CurveFitTool 的项目属性中添加对 SyslabAppSdk 的附加库目录依赖：属性→链接器→常规→附加库目录，添加：\$(OutDir)，如下图所示：

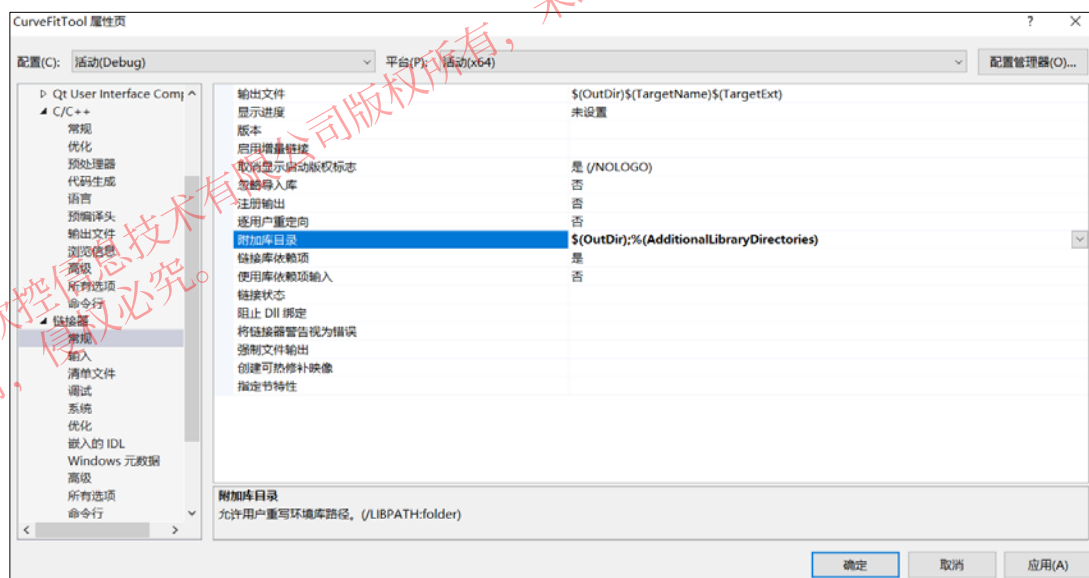


图 3-13 附加库目录

(5) 在 CurveFitTool 的项目属性中添加对 SyslabAppSdk 的附加依赖项：属性→链接器→输入→附加依赖项，添加：SyslabAppSdk.lib，如下图所示：

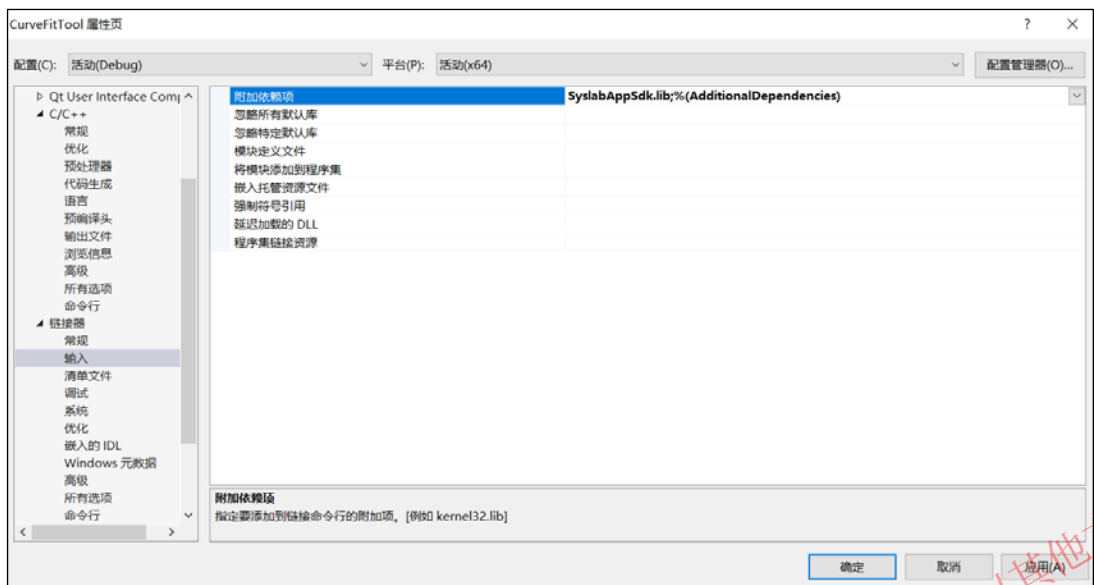


图 3-14 附加依赖项

(6) 生成 CurveFitTool 的项目依赖项，选中 CurveFitTool 项目后右键菜单选择：生成依赖项→项目依赖项→依赖于 SyslabAppSdk，如下图所示：

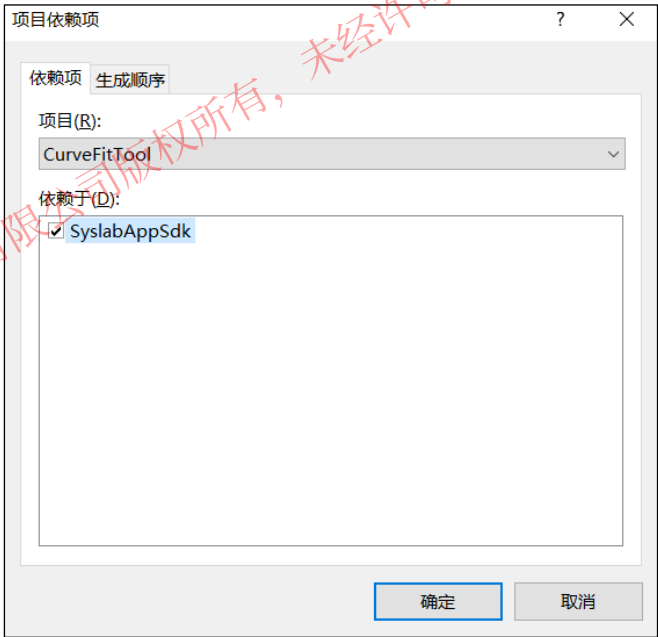


图 3-15 配置项目依赖

到这一步，依赖 SyslabAppSdk 的配置就完成了。

3.2.1.2.2 传递 APP 启动参数

Syslab 平台启动 APP 程序时，会传递给 APP 进程以下启动参数：

- argv[0]：用于调用程序的命令；

- argv[1]: 管道名, 由 Syslab 平台生成后传递给 APP;
- argv[2~3]: Julia 相关路径参数, 由 Syslab 平台生成后传递给 APP;
- argv[4~]: 由用户在 APP 启动函数中输入。

还是以 CurveFitTool 为例, 在 main() 启动函数中负责解析这些参数, 其中管道名是必须解析的, julia 参数和其他参数 APP 可以按需解析和使用。示例代码如下:

```
// 获取管道名
if (argc > 1 && strcmp(argv[1], "--pipe=") >= 0)
{
    QString pipe_name = QString(argv[1]);
    pipe_name = pipe_name.replace("--pipe=", "");
    w.SetPipeName(pipe_name);
}

// 获取 julia 参数
/*
if (argc > 3 && strcmp(argv[2], "--julia_bindir=") >= 0 && strcmp(argv[3],
"--image_path=") >= 0)
{
    QString julia_bindir = QString(argv[2]);
    julia_bindir = julia_bindir.replace("--julia_bindir=", "");
    w.SetJuliaBinDir(julia_bindir);

    QString image_path = QString(argv[3]);
    image_path = image_path.replace("--image_path=", "");
    w.SetSysImagePath(image_path);
}
*/

// 获取多项式指数
if (argc > 4)
{
    QString qstr_term = QString::fromLocal8Bit(argv[4]);
    w.SetTerm(qstr_term);
}
```

以管道名参数为例, 解析后的管道名要传递给 QMainWindow 的实现类 CurveFitTool, 在 CurveFitTool 中新增函数 SetPipeName, 示例代码如下:

- curve_fit_main_win.h

```
#include <QtWidgets/QMainWindow>
#include "ui_curve_fit_main_win.h"
class SyslabAppSdk;
class CurveFitMainWin : public QMainWindow
{
    Q_OBJECT
public:
    CurveFitMainWin(QWidget *parent = nullptr);
    ~CurveFitMainWin();
    void SetPipeName(const QString& pipe_name);
private:
    Ui::CurveFitToolClass ui;
    SyslabAppSdk* m_syslabSdk; //Syslab SDK 接口
};
```

● curve_fit_main_win.cpp

```
#include "curve_fit_main_win.h"
#include "syslab_app_sdk.h"
CurveFitMainWin::CurveFitMainWin(QWidget *parent)
    : QMainWindow(parent)
{
    ui.setupUi(this);
}
CurveFitMainWin::~CurveFitMainWin()
{}
void CurveFitMainWin::SetPipeName(const QString& pipe_name)
{
    if (m_syslabSdk == nullptr)
    {
        m_syslabSdk = new SyslabAppSdk("CurveFitTool", pipe_name.toStdString());
    }
}
```

其他参数以类似的方法可以传递给 CurveFitTool，在此不再赘述。

进一步开发 UI 界面，UI 界面的开发方法在此不做说明。开发好的 UI 界面效果如下图所示：

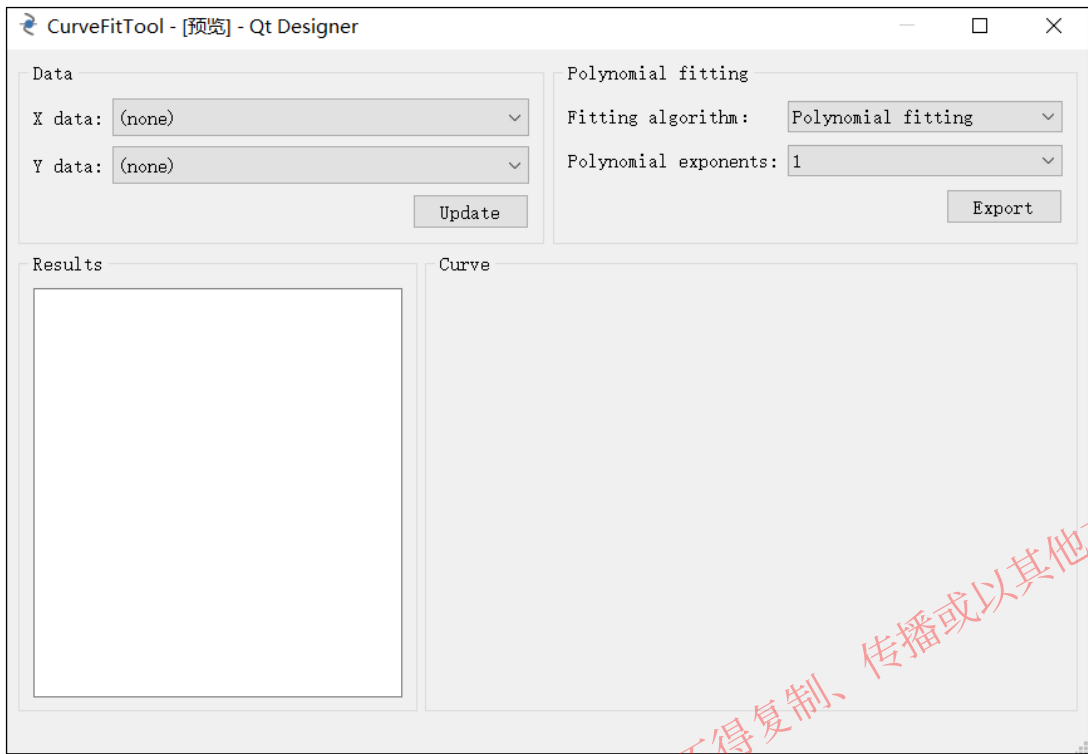


图 3-16 UI 界面设计

以上界面交互中，“Data”数据部分需要从 Syslab 工作空间获取数组类型的变量名列表，以及通过变量名获取变量值；“Export”导出部分需要将计算结果写入 Syslab 工作空间。

SyslabAppSdk 主要提供以下接口：

- 获取变量列表
- 查询变量值
- 发送脚本到 Syslab 执行

下文将讲述这些接口的具体使用方法。

3.2.1.2.3 获取变量列表

<pre>bool MwGetVariables(bool show_modules, std::vector<VariableInfo>& variables)</pre>	
功能	获取 Syslab 工作区变量列表
参数	[in] show_modules 是否显示模块列表，一般为 false [out]variables 变量名列表
返回值	true-成功，false-失败

CurveFitTool 中调用该接口的示例代码如下：

```
vector<VariableInfo> variables;
if (m_syslabSdk)
{
    m_syslabSdk->MwGetVariables(false, variables);
}
for (VariableInfo var : variables)
{
    QString var_type = QString::fromStdString(var.GetType());
    if (var_type.contains("Vector{Int64}")
        || var_type.contains("Vector{Float64}")
    )
    {
        ui.comboBox_x->addItem(QString::fromStdString(var.GetName()));
        ui.comboBox_y->addItem(QString::fromStdString(var.GetName()));
    }
}
```

3.2.1.2.4 获取变量值

```
bool MwGetValue(const std::string& var, std::string& value);
```

功能	获取工作区变量值
参数	[in] var 变量名，可以为子变量 a.b [out] value 变量值（字符串形式）
返回值	true-成功，false-失败

CurveFitTool 中调用该接口的示例代码如下：

```
string value;
m_syslabSdk->MwGetValue(var.toStdString(), value);
QString qstr_value = QString::fromStdString(value);
```

3.2.1.2.5 运行脚本

```
std::string MwRunScript(const std::string& code,
    bool show_code_in_repl = false,
    bool show_result_in_repl = false)
```

功能	在 syslab 工作区执行 Julia 脚本代码
参数	[in] code 要运行的 julia 脚本

	[in] show_code_in_repl 是否在 Syslab REPL 中显示代码 [in] show_result_in_repl 是否在 Syslab REPL 中显示结果
返回值	code 脚本运行后的结果

CurveFitTool 中调用该接口输出结果到 syslab 环境的示例代码如下：

```
if (dlg.IsExportRes())
{
    QString str = dlg.GetResName() + "=" + ListToStr(m_yResDatas);
    m_syslabSdk->MwRunScript(str.toStdString().c_str());
}
```

调用 Syslab 的数学 API 进行曲线拟合计算的示例代码如下，第一步计算曲线拟合参数：

```
// p = vec(polyfit(cdate, pop, 4))
QString str = QString("import TyMathCore;
    curvefit_coefficient, cf_s = TyMathCore.polyfit(%1, %2, %3)")
    .arg(x_datas_str).arg(y_datas_str).arg(n);
m_syslabSdk->MwRunScript(str.toStdString().c_str());
string res;
m_syslabSdk->MwGetValue("curvefit_coefficient", res);
```

第二步计算曲线拟合结果：

```
// res = polyval(p, cdate)
QString str = QString("import TyMathCore;curvefit_res =
TyMathCore.polyval(curvefit_coefficient, %1)").arg(x_datas_str);
m_syslabSdk->MwRunScript(str.c_str(), true, true);
String res;
m_syslabSdk->MwGetValue("curvefit_res", res);
```

3.2.1.2.6 关闭命名管道

~SyslabAppSdk()	
功能	发送请求，关闭命名管道，在关闭 APP 程序前调用
参数	无
返回值	无

在关闭 APP 前，需要调用 SyslabAppSdk 的析构函数，与 Syslab 平台断开连接。CurveFitTool 中调用该接口的示例代码如下：

- curve_fit_main_win.cpp

```
CurveFitMainWin::~CurveFitMainWin()
{
    if (m_syslabSdk != nullptr)
    {
        delete m_syslabSdk;
    }
}
```

到此为止，曲线拟合工具开发过程中要调用 SDK 的接口都已经讲完了，剩下的就是具体的业务逻辑代码开发。

该示例 Demo 的源代码，可以在 Syslab 安装软件以下路径获取到：<Syslab 安装路径>\Examples\10 AppDemos\cpp\。

3.2.2 APP 测试

3.2.2.1 打桩测试

以曲线拟合 APP 为例，该 APP 在运行时从 Syslab 平台获取数据后，进行曲线拟合计算，对 Syslab 平台有依赖。测试时为了解除对 Syslab 平台的依赖，可以通过打桩 mock 的方式，构造打桩数据，独立验证 APP 自身的功能。

打桩测试代码如下：

```
bool mock = true;
void CurveFitMainWin::SlotUpdateSyslabVar()
{
    vector<VariableInfo> variables;
    if (mock)
    {
        VariableInfo var1 = VariableInfo("cdate", "Vector{Float64}");
        VariableInfo var2 = VariableInfo("pop", "Vector{Float64}");
        variables.push_back(var1);
        variables.push_back(var2);
    } else if (m_syslabSdk)
    {
        m_syslabSdk->MwGetVariables(false, variables);
    }
    //其他代码...
}

void CurveFitMainWin::SlotUpdateXData(QString var)
```

```
{
    string value;
    if (mock)
    {
        value
"[1790,1800,1810,1820,1830,1840,1850,1860,1870,1880,1890,1900,1910,1920,1930,1940,1950,1960,1
    }
    if (m_syslabSdk && var != NONE)
    {
        m_syslabSdk->MwGetValue(var.toStdString(), value);
    }
    //其他代码...
}

void CurveFitMainWin::SlotUpdateYData(QString var)
{
    string value;
    if (mock)
    {
        value
"[3.9,5.3,7.2,9.6,12.9,17.1,23.1,31.4,38.6,50.2,62.9,76.0,92.0,105.7,122.8,131.7,150.7,179.0,
    } else if (m_syslabSdk && var != NONE)
    {
        m_syslabSdk->MwGetValue(var.toStdString(), value);
    }
    //其他代码...
}
```

3.2.2.2 集成测试

APP 开发和打包完成后，参考[章节 2.3](#)的内容，完成 APP 的集成测试。

3.2.3 APP 打包

将本程序所依赖的 Qt 库放入文件夹，依赖 Qt 库如下：

- Qt5Charts.dll
- Qt5Core.dll
- Qt5Gui.dll
- Qt5widgets.dll

将本程序依赖的 SyslabSDK 库放入文件夹：

- SyslabAppSdk.lib

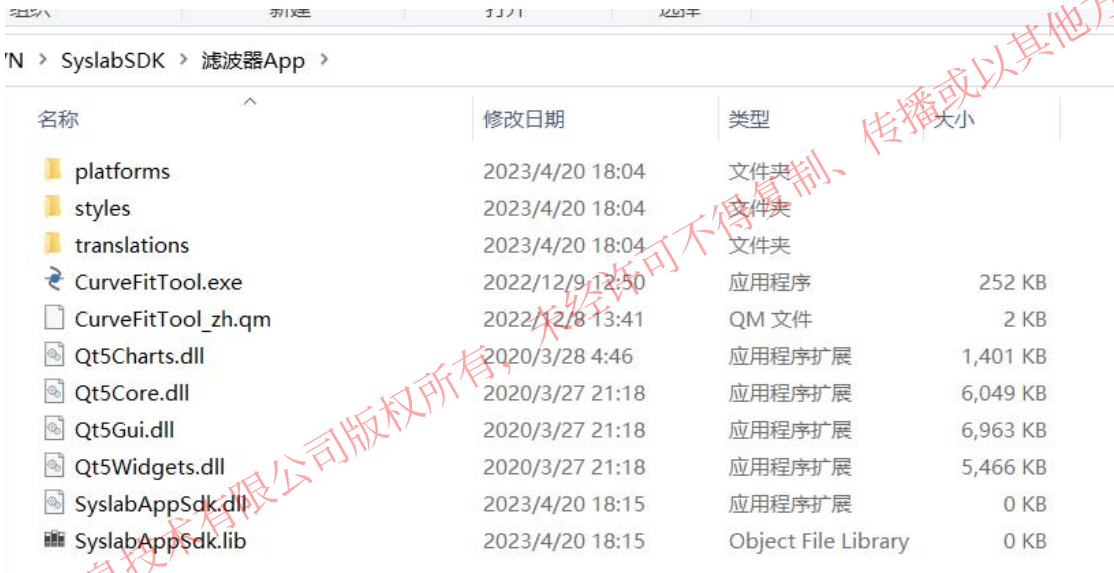
放入生成的 exe 及其依赖的相关翻译文件：

- CurveFitTool.exe
- CurveFitTool_zh.qm
- translations(翻译文件夹)

windows 相关动态库：

- platforms/qwindows.dll
- styles/qwindowsvistastyle.dll

将以上所有文件打包成如下文件夹：



名称	修改日期	类型	大小
platforms	2023/4/20 18:04	文件夹	
styles	2023/4/20 18:04	文件夹	
translations	2023/4/20 18:04	文件夹	
CurveFitTool.exe	2022/12/9 12:50	应用程序	252 KB
CurveFitTool_zh.qm	2022/12/8 13:41	QM 文件	2 KB
Qt5Charts.dll	2020/3/28 4:46	应用程序扩展	1,401 KB
Qt5Core.dll	2020/3/27 21:18	应用程序扩展	6,049 KB
Qt5Gui.dll	2020/3/27 21:18	应用程序扩展	6,963 KB
Qt5Widgets.dll	2020/3/27 21:18	应用程序扩展	5,466 KB
SyslabAppSdk.dll	2023/4/20 18:15	应用程序扩展	0 KB
SyslabAppSdk.lib	2023/4/20 18:15	Object File Library	0 KB

图 3-17 C++ APP 打包